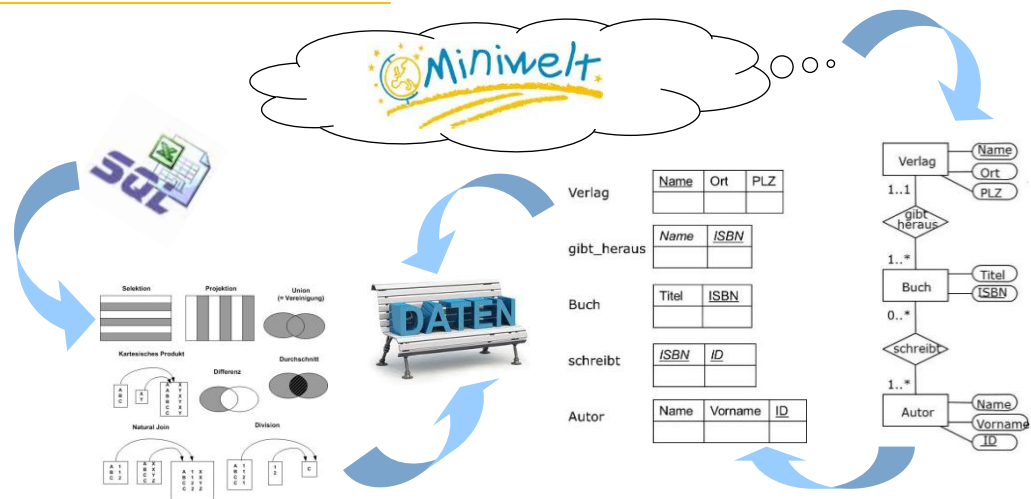
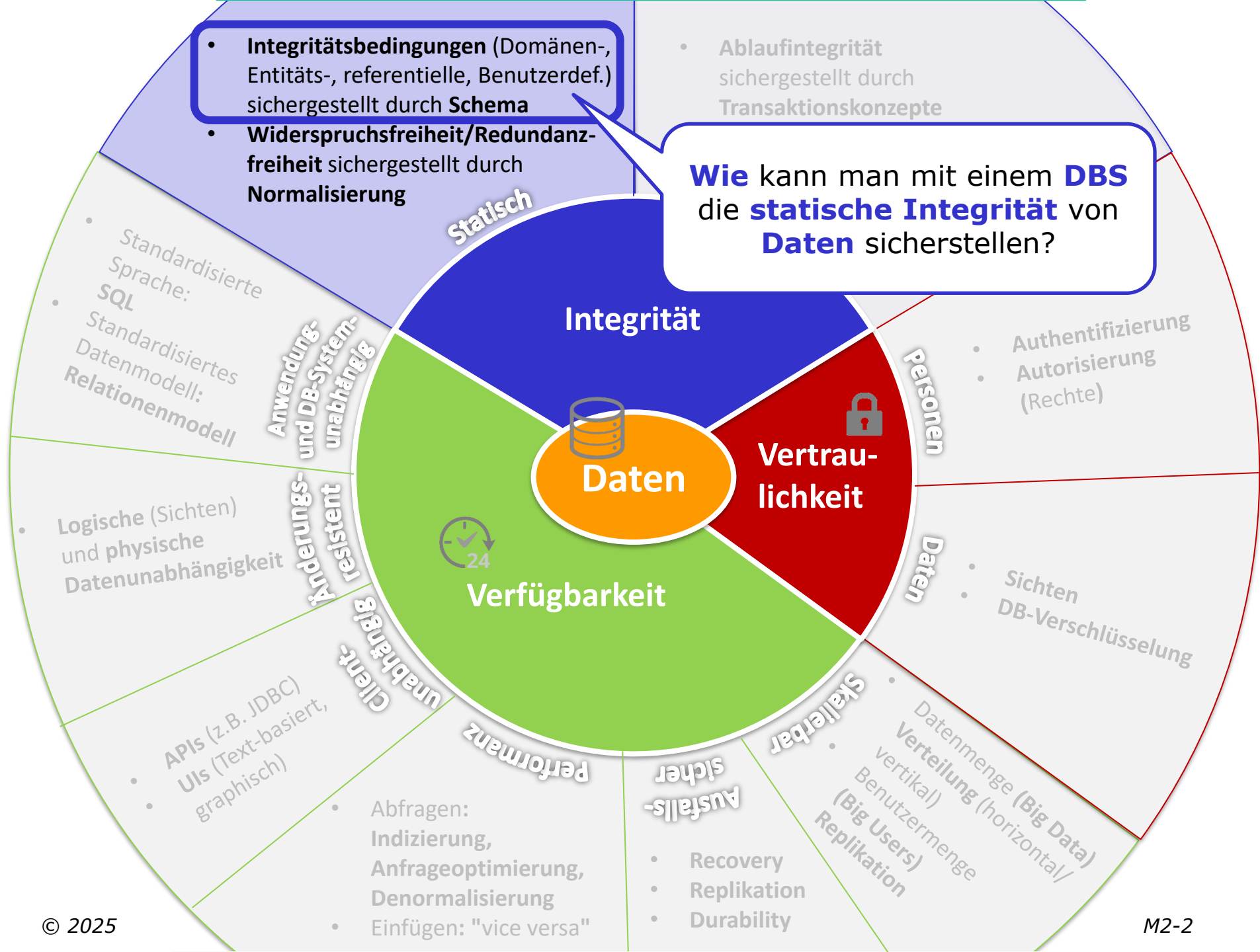


Datenbankentwurf

Josef Altmann





Probleme mit Datenhaltung mit Dateien

Datenbanksysteme bieten **formale Sprache**, um DB-Schemata zu definieren

DB-Schema

- = **Integritätsbedingungen** (Regeln), die sicherstellen, dass die Daten in einem **gültigen Zustand** sind
- Daten, die diesen **Regeln widersprechen**, können **nicht eingefügt** werden

■ Integrität der Daten?

z.B. Vermeidung von

- **Mehrfachspeicherung** (Redundanz)
- **Widersprüchlichen Daten** (Konsistenz)
- **unterschiedlichem Aufbau** (Struktur)
z.B. in **Anzahl** (4 statt 3 Attribute)
bzw. **Art** (z.B. Datentyp)

müssen entsprechen

ISBN-10; Titel; Standort

0132350882;"Clean Code: ...";1E35
 0132350882;"Clean Code: ...";1E34
 "The Clean Coder"; 0137081073;2I64;Foo
 ...

Bücher.csv

DB-Schema

DB-Schema spielt eine **analoge Rolle** wie eine **Grammatik** bei einer **Programmiersprache** → **Programm** muss **gewissen Regeln** entsprechen, um **statisch** (zur Kompilierzeit) **gültig** zu sein!

?

DB-Schema

müssen entsprechen

ISBN-10; Titel; Standort

0132350882;"Clean Code: ...";1E34

0137081073,"The Clean Coder";2I64

...

Bücher.csv
Daten

```
Program → <?php Block ?>
Block → { funcDef } Statements
Statements → { Statement }
funcDef → function Identifier ( Identifier { , Identifier } ) { Statements
        { return ListExpr ; } }
Statement → if ( Expression ) Statement [ else Statement ]
           | while ( Expression ) Statement
           | foreach ( Identifier as Identifier ) Statement
           | Identifier [ [ [ Expression ] ] = ListExpr ;
           | array push ( Identifier , AddExpr ) ;
           | print AddExpr ;
           | { Statement { Statement } }
Expression → AndExpr { || AndExpr }
AndExpr → RelExpr { && RelExpr }
```

Grammatik

muss entsprechen

```
<?
$dbc=odbc_connect("gbook","","");
if (!$dbc)
{exit("Connection Failed: " . $dbc);}
$query="SELECT * FROM comments";
$rs=odbc_exec($dbc,$query);
if (!$rs)
{exit("Error in SQL");}
echo '<h3>MS Access powered Guest Book</h3>';
while (odbc_fetch_row($rs))
{
    $e_name=odbc_result($rs,"name");
    $comment=odbc_result($rs,"comment");
    $e_date=odbc_result($rs,"entry_date");

    echo '<b>Name:</b>'. $e_name. '<br>';
```

Programm

Aufgaben Grammatik vs. DB-Schema

Grammatik-Regeln

Definieren **alle erlaubten Konstrukte** (z.B., Variablendeklarationen, Statements wie `if`, `while`, `for`,...)

Definieren, **in welcher Anordnung (Beziehung)** diese Konstrukte vorkommen dürfen (z.B. nach einem `if` muss eine boolsche Bedingung folgen)

Müssen für **alle möglichen Programme** gelten, um gültige Programme von ungültigen Programmen unterscheiden zu können

Müssen in einer **formalen Sprache** (häufig verwendet: EBNF) definiert sein, damit sie **maschineninterpretierbar** sind

Aufgaben Grammatik vs. DB-Schema

Grammatik-Regeln	DB-Schema-Regeln
Definieren alle erlaubten Konstrukte (z.B., Variablendeklarationen, Statements wie <code>if</code> , <code>while</code> , <code>for</code> ,...)	Definieren alle gültigen "Datenobjekte" (z.B., Vorlesungen, Studenten,...)
Definieren, in welcher Anordnung (Beziehung) diese Konstrukte vorkommen dürfen (z.B. nach einem <code>if</code> muss eine boolsche Bedingung folgen)	Definieren, in welcher Beziehung die "Datenobjekte" stehen können (z.B., Studenten <u>besuchen</u> Vorlesungen,...)
Müssen für alle möglichen Programme gelten, um gültige Programme von ungültigen Programmen unterscheiden zu können	Müssen für alle möglichen Daten gelten, um gültige Daten von ungültigen Daten unterscheiden zu können
Müssen in einer formalen Sprache (häufig verwendet: EBNF) definiert sein, damit sie maschineninterpretierbar sind	Müssen mit einer formalen Sprache (häufig verwendet: SQL DDL) definiert sein, damit sie maschineninterpretierbar sind
	Sollen auch noch Redundanz vermeiden und damit potentielle Widersprüche in den Daten

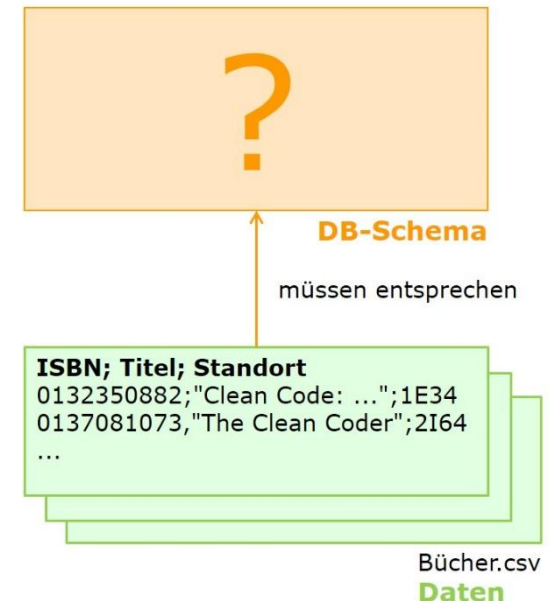
⇒ **Programme**, die nicht den Grammatik-Regeln genügen, können **nicht kompiliert** werden

⇒ **Daten**, die nicht den DB-Schema-Regeln genügen, können **nicht in die DB eingefügt** werden

Herausforderungen bei der Erstellung eines DB-Schemas

■ Zu **berücksichtigende Aspekte**

- Welche **Arten** von **Datenobjekten** sollen überhaupt erfasst werden bzw. **welche nicht**? D.h., welche Daten sind relevant bzw. nicht relevant?
- Welche **Eigenschaften** charakterisieren diese Datenobjekte?
- In welchen **Beziehungen** können diese Datenobjekte stehen?
- Wie kann ich sicherstellen, dass die **Daten nicht redundant** gehalten werden?



- **Zu viele Aspekte**, um ein DB-Schema auf einen Schlag in einer **formalen Sprache** niederzuschreiben
- **Systematischer Prozess** für den **Datenbankentwurf** notwendig!

Überblick

- Datenbankentwurfsschritte
- Modellbegriff
- Entity-Relationship-Modell
- Erweiterungen des Entity-Relationship-Modells
- Designprinzipien
- Anhang I: Definitionen

Datenbankentwurfsschritte

Systemvision

Anforderungsanalyse

Phase	Modell	Ergebnis
Anforderungsanalyse	Requirements Engineering	Lasten-/Pflichtenheft (Use-Cases, User Stories, ...) <i>Realitätsausschnitt</i> ANF 5.4: Eine Vorlesung wird durch eine eindeutige Vorlesungsnummer identifiziert. ...
Konzeptueller Entwurf	Entity Relationship (UML, ...)	Konzeptuelles DB-Schema
Logischer Entwurf	Datenbankmodell (hierarchisch, objekt-orientiert, objekt-relational, semi-strukturiert, ...)	Logisches DB-Schema Vorlesung{ <u>VorlNr</u> , Titel, ...} Student{ <u>MatrNr</u> , Name, ...}
Physischer Entwurf	Data Definition Language	Physisches DB-Schema <pre> CREATE TABLE Vorlesung(VorlNr NUMBER(10) PRIMARY KEY, Titel VARCHAR(20), ...); </pre>



Lauffähiges System

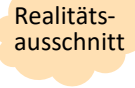
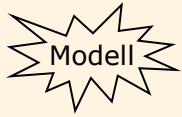
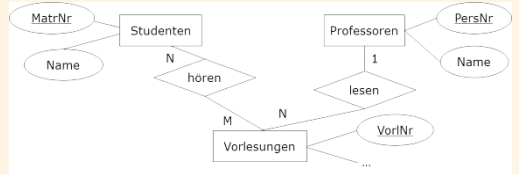
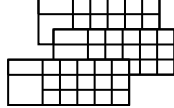
Anforderungsanalyse



- In dieser ersten Phase wird der **Bedarf** an **Informationen** gesammelt und **analysiert**
- Die **Ergebnisse** werden vorwiegend in **informalen Beschreibungen** (Texte, tabellarische Aufstellungen, Formblätter usw.) des Fachproblems gesammelt
- Als **Methoden** zur Ermittlung des Informationsbedarfs (Nutzerforschung) werden
 - Interviews, Beobachtung, Fokusgruppen, Nutzerstudien
 - die Analyse bestehender Arbeitsprozesse,
 - sowie die Analyse existierender Dokumente (z.B. Formulare) eingesetzt
- **Ergebnisse**: Pflichtenheft, User Stories, Use Cases, (explorative, evolutionäre) Prototypen, **Datenmodell**, ...

Datenbankentwurfsschritte

Konzeptuelle Modellierung (Abstraktion)

Phase	Modell	Ergebnis
Anforderungsanalyse	Requirements Engineering	Pflichtenheft (Use-Cases, ...)  ANF 5.4: Eine Vorlesung wird durch eine eindeutige Vorlesungsnummer identifiziert. ...
Konzeptueller Entwurf	Entity Relationship	Konzeptuelles DB-Schema  
Logischer Entwurf	Datenbankmodell (hierarchisch, objekt-orientiert, objekt-relational, semi-strukturiert, ...)	Logisches DB-Schema  <pre> Vorlesung{<u>VorlNr</u>, Titel, ...} Student{<u>MatrNr</u>, Name, ...} </pre>
Physischer Entwurf	Data Definition Language	Physisches DB-Schema  <pre> CREATE TABLE Vorlesung(VorlNr NUMBER(10) PRIMARY KEY, Titel VARCHAR(20), ...); </pre>

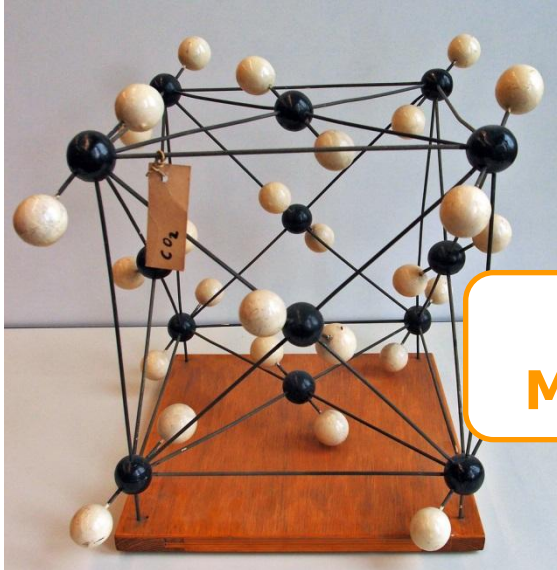
Konzeptueller Entwurf

- Im **konzeptuellen Entwurf** wird die **erste FORMALE Beschreibung** des **Systems** erstellt; diese abstrahiert aber noch von zahlreichen Details wie z.B. Implementierungsdetails („frei von Technik!“)
- Als **Beschreibungsmittel** wird ein **abstraktes und formales konzeptuelles Modell** eingesetzt, wie beispielsweise ein **Entity-Relationship-Modell**

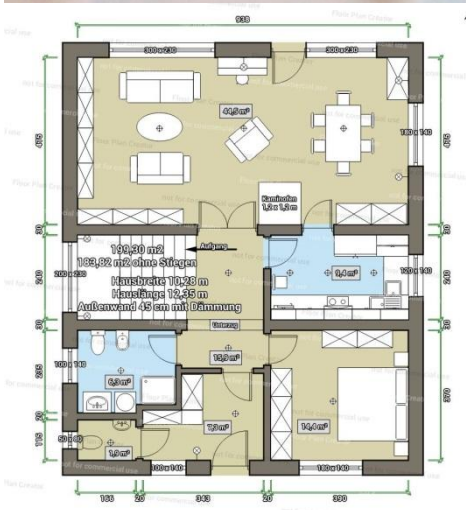
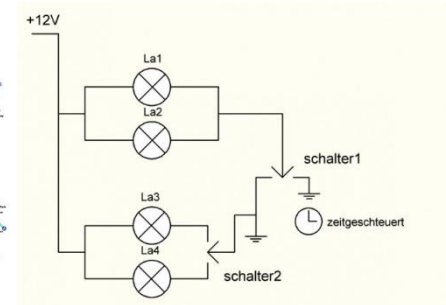
Was ist ein **Modell**?

Modelle

$$f(u, v) = \begin{cases} |u-v| > \Delta_{uv} \wedge u > v \wedge v \geq 0 \Rightarrow V_{motor} \\ |u-v| > \Delta_{uv} \wedge u > v \wedge v < 0 \Rightarrow V_{dremis} \\ |u-v| > \Delta_{uv} \wedge u \leq v \wedge v \geq 0 \Rightarrow -V_{dremis} \\ |u-v| > \Delta_{uv} \wedge u \leq v \wedge v < 0 \Rightarrow -V_{motor} \\ |u-v| \leq \Delta_{uv} \wedge u \geq v \wedge v \geq 0 \Rightarrow \frac{V_{motor}}{\Delta_{uv}} \cdot (u-v) \\ |u-v| \leq \Delta_{uv} \wedge u \geq v \wedge v < 0 \Rightarrow \frac{V_{dremis}}{\Delta_{uv}} \cdot (u-v) \\ |u-v| \leq \Delta_{uv} \wedge u < v \wedge v \geq 0 \Rightarrow \frac{V_{dremis}}{\Delta_{uv}} \cdot (u-v) \\ |u-v| \leq \Delta_{uv} \wedge u < v \wedge v < 0 \Rightarrow \frac{V_{motor}}{\Delta_{uv}} \cdot (u-v) \end{cases}$$



Was haben diese
Modelle gemeinsam?



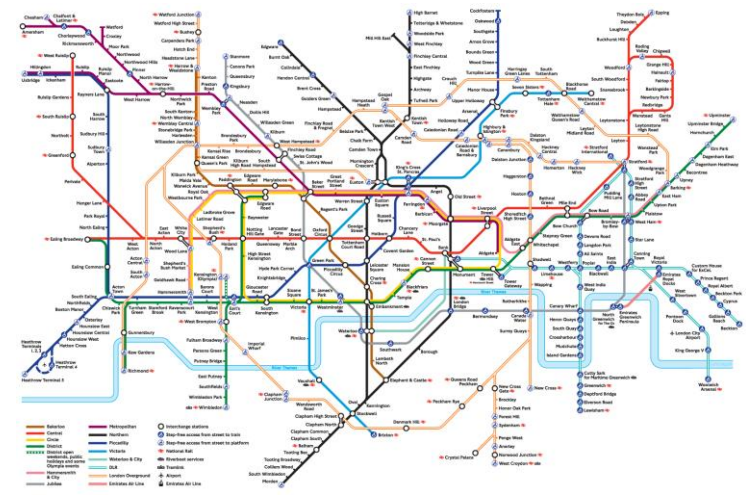
Was bitte ist ein Orrery?

Was ist ein Modell?

- Ein **Modell** ist ein **vereinfachtes Abbild** der **Wirklichkeit**, d.h.,
 - Modelle erfassen meist **nicht** alle Individuen und Attribute (sondern sind eine **Verkürzung/Verdichtung** = **Abstraktion**)
 - Es wird nur das modelliert, was dem Modellierer für **wichtig/nützlich/notwendig** im Verwendungskontext erscheint
 - Modelle haben immer einen bestimmten Zweck



Realitätsausschnitt



Modell

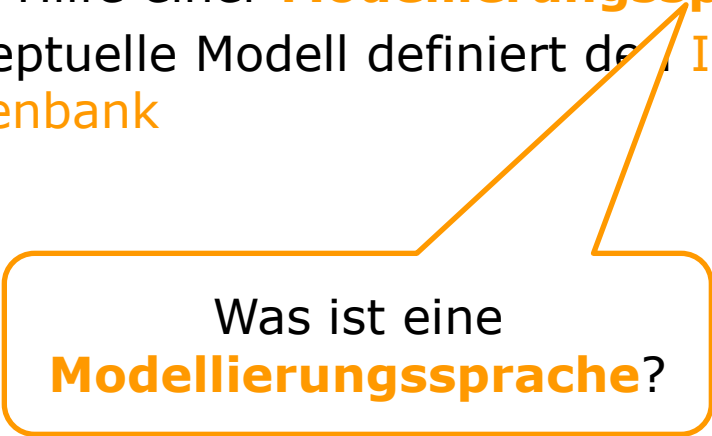
Konzeptuelles Modell

■ Abstraktion

- Es wird ein bestimmter Realitätsausschnitt im benötigten Detailgrad abgebildet

■ Zweck

- Das Konzeptuelle Modell beschreibt die Struktur von Daten mit Hilfe einer **Modellierungssprache**
- Das Konzeptuelle Modell definiert den Informationsgehalt einer Datenbank



Was ist eine
Modellierungssprache?

Modellierungssprache

- Modellierungssprachen erlauben die **Anforderungen** an ein **Softwaresystem** in Bezug auf **Struktur** (und Verhalten) auf einer **höheren Abstraktionsebene** festzulegen
- **Beispiele für Modellierungssprachen:**
 - **Entity-Relationship (ER)** → in dieser LVA verwendet – erlaubt Modellierung von Struktur
 - Unified Modeling Language (UML) – erlaubt Modellierung von Struktur und Verhalten
- Modellierungssprache ist eine
 - **formale Sprache** (Syntax und Semantik)
 - ist ein Hilfsmittel zum Erreichen der **Datenabstraktion**
 - ist **unabhängig** vom eingesetzten **Datenbanksystem**

Überblick

- Datenbankentwurfsschritte
- Modellbegriff
- Entity-Relationship-Modell
- Erweiterungen des Entity-Relationship-Modells
- Designprinzipien
- Anhang I: Definitionen

Entity-Relationship-Modell

Allgemeines



Peter Chen

- Das Entity-Relationship-Modell wurde 1976 in einem wegweisenden Aufsatz von **Peter Chen** vorgestellt.
- Wie der Name schon ausdrückt, modelliert dieses Modell **Gegenstände (Entities)** und die **Beziehungen (Relationships)** zwischen diesen.
- Notation wurde später erweitert $\Rightarrow$ **EER-Modell**
- Viele **unterschiedliche graphische Notationen** - Konzepte (Entity, Relationship, Attribut, Schlüssel, Rolle) sind jedoch überall dieselben.

Entity-Relationship-Modell

Drei Grundkonzepte

■ Entity

- **Objekt** der realen oder der Vorstellungswelt, über das Informationen zu speichern sind, z.B. *Vorlesung*, *Professor*
- aber auch Informationen über **Ereignisse**, wie z.B. *Prüfung*

■ Relationship

- beschreibt eine **Beziehung** zwischen Entities, z.B. ein Professor *liest* eine Vorlesung oder ein Student *hört* eine Vorlesung

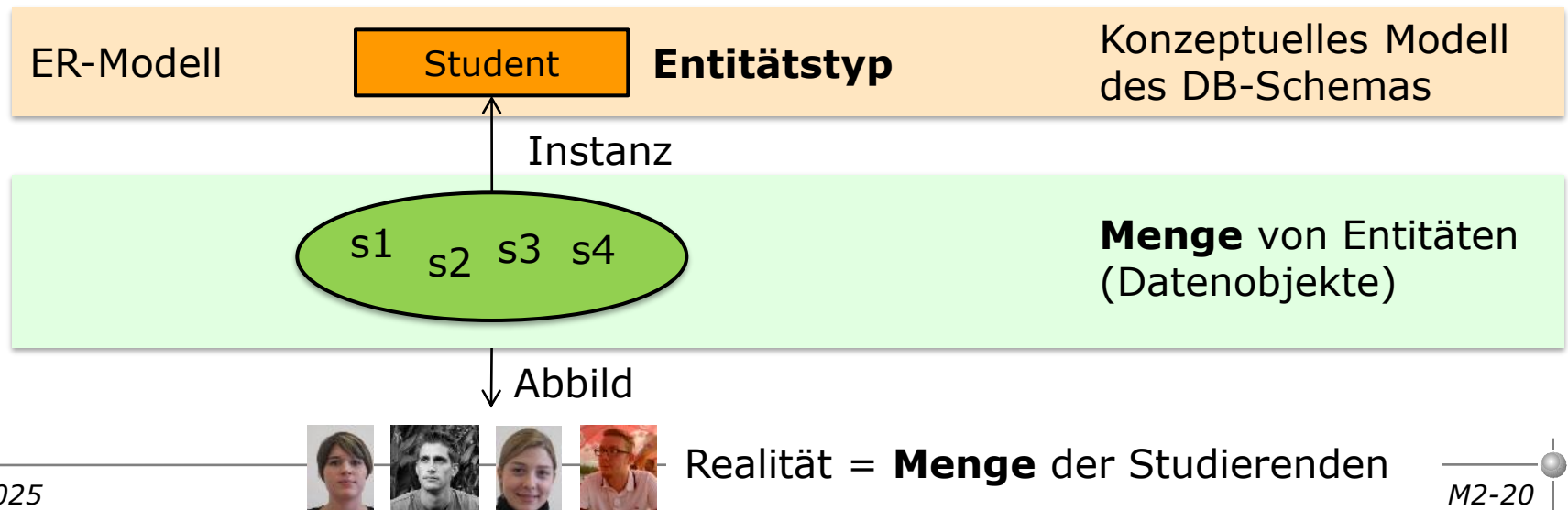
■ Attribut

- repräsentiert eine **Eigenschaft** von Entities oder Beziehungen, z.B. *Name* eines Professors, *Titel* einer Vorlesung



Entitätstyp

- **ER-Konzept:** Entitätstyp
- **Notation:** `<<Name>>`
- **Semantik:** Ein Entitätstyp beschreibt eine **Menge** von **Entitäten**, welche **gleiche Attribute/Merkmale** aufweisen; eine Entität ist ein **eindeutig** zu bestimmendes **Objekt**, über das **Informationen gespeichert** werden soll; das Objekt kann materiell oder immateriell, konkret oder abstrakt sein
- **Beispiel ER-Modell:**



Entitätstyp

Checkliste zur Festlegung von Entitätstypen

- Lassen sich konkrete Objekte identifizieren?
 - Bei techn. Systemen: Reale Objekte/Dinge sind Ausgangspunkt.
 - Bei kommerziellen Systemen: Formulare als Vorlage
- Zu welchen Kategorien gehören die Entitätstypen?
 - Konkrete Objekte
 - Personen und deren Rollen
 - Informationen über Aktionen, wie z.B. eine Banküberweisung
- Liegt ein aussagefähiger Entitätstypname vor?
- Wann liegt kein Entitätstyp vor?
 - Es lassen sich weder Attribute noch Beziehungen identifizieren.
 - Entity-Typ enthält dieselben Attribute wie ein anderer Entitätstyp.
- Benennung von Entitätstypen im Plural oder Singular?
 - Singular bevorzugen

Attribute

- **ER-Konzept:** Attribut

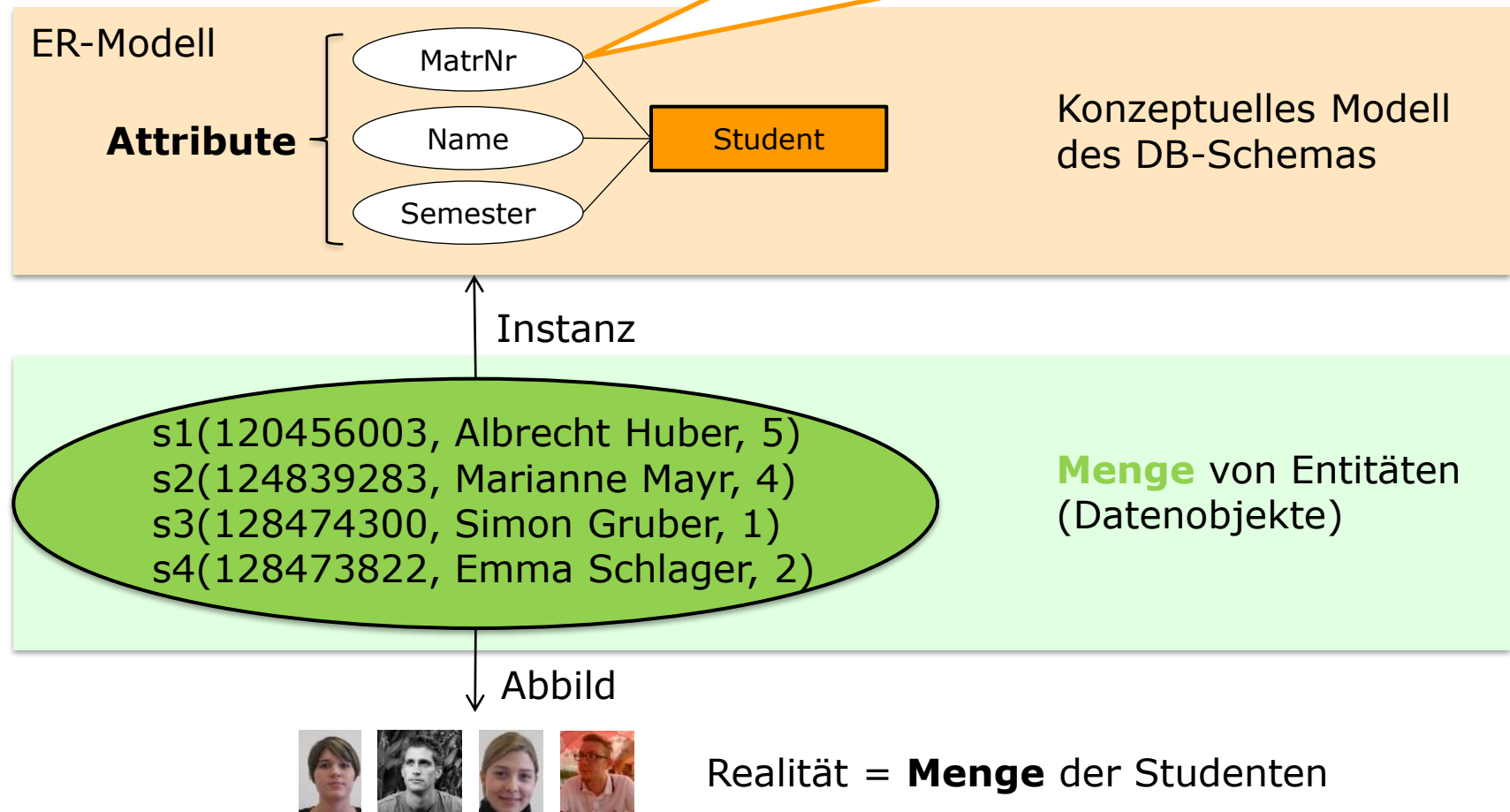
- **Notation:** 

- **Semantik:** Attribute modellieren **Eigenschaften** von Entitäten (oder auch Beziehungen – siehe später); da alle Entitäten eines Entitätstyps dieselben Arten von Eigenschaften haben, werden Attribute für Entitätstypen deklariert

Attribute

■ Beispiel ER-Modell

Attribute weisen einen **Wertebereich** (Datentyp, z.B. int, string, date) auf; auf **Instanzebene** muss der **Wert** aus diesem **Wertebereich** kommen



Attribute

Checkliste zur Festlegung von Attributen

- Ist ein Attribut problemrelevant?
- Ist der Attributname geeignet?
 - sprechende Namen, CamelCase (bspw. ProduktId)
 - Eindeutigkeit innerhalb des Entity-Typs
 - keine Umlaute, Sonderzeichen, Leerzeichen, nicht mit Zahlen beginnen
- Welche Datentypen liegen vor?
 - Standarddatentypen: NUMBER, DATE, VARCHAR
- Liegen für Attribute bestimmte Domänen, das heißt festgelegte Wertebereiche vor?
 - Wochentag mit Wertebereich {Montag, ..., Freitag}
- Gehört ein Attribut zu einem Entitätstyp oder zu einer Beziehung?
- Welche Attribute können als Schlüsselattribute dienen?

Schlüssel

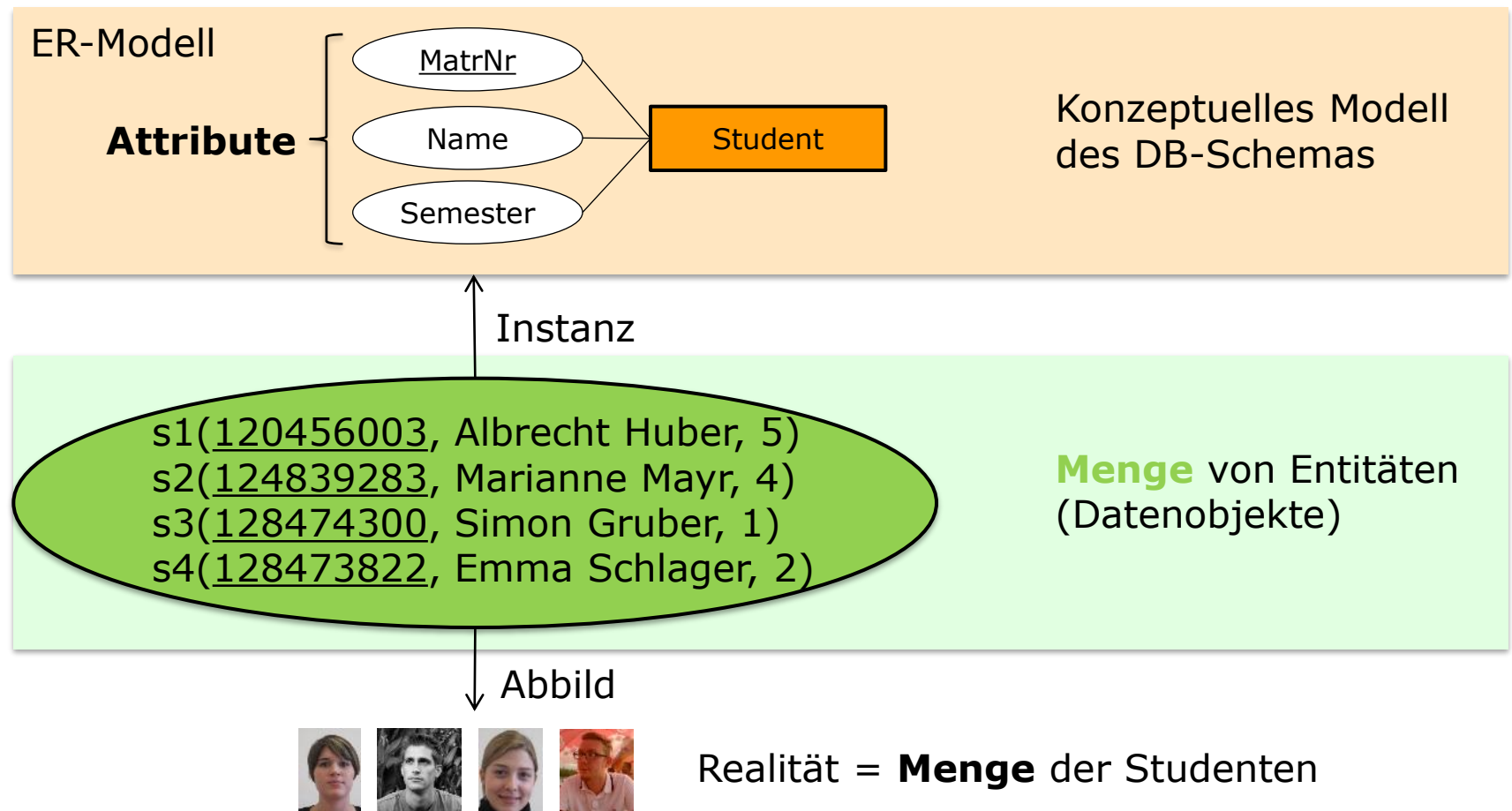
- **ER-Konzept:** Schlüssel

- **Notation:**

- **Semantik:** Ein Schlüssel ist eine **minimale Teilmenge** der **Attribute** eines Entitätstyps, dessen **Werte** für eine **Entität** **eindeutig** (identifizierend) sind; bei mehreren potentiellen Schlüsseln wird ein Schlüssel ausgewählt (= **Primärschlüssel**)

Schlüssel

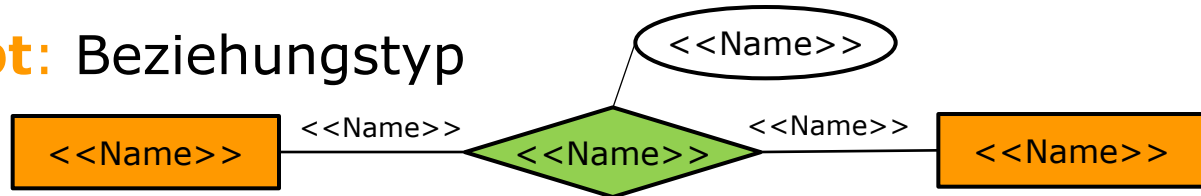
■ Beispiel ER-Modell



Beziehungstyp

■ ER-Konzept: Beziehungstyp

■ Notation:

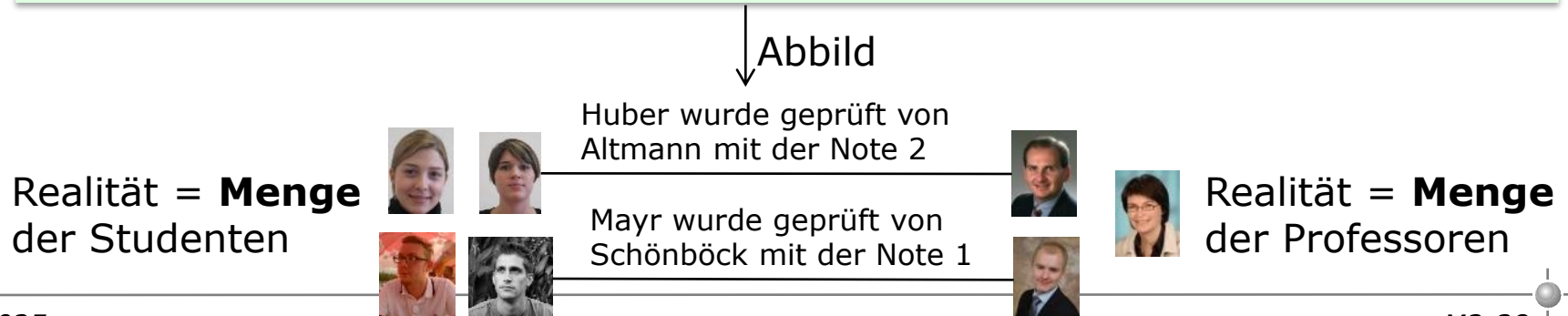
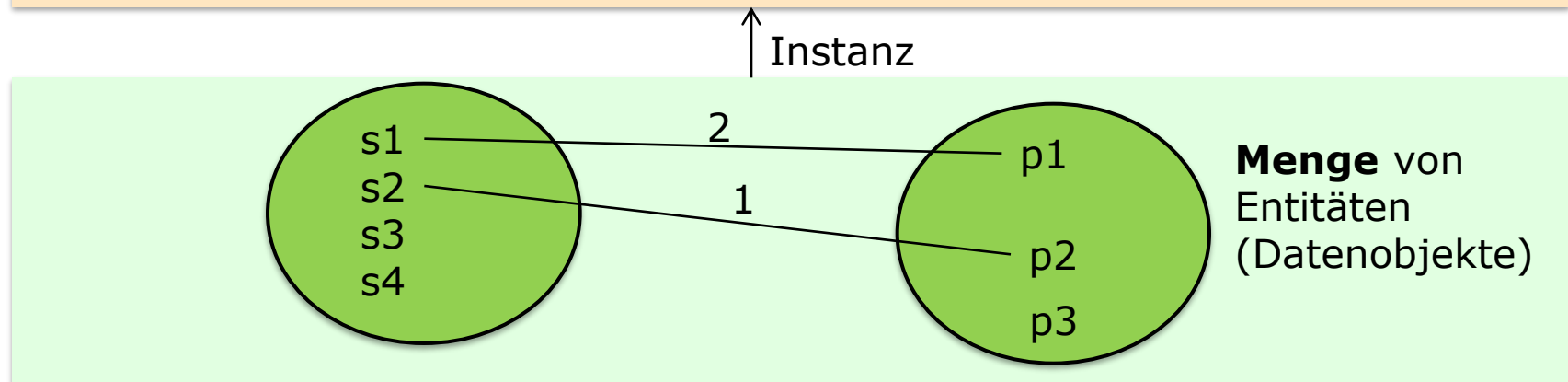
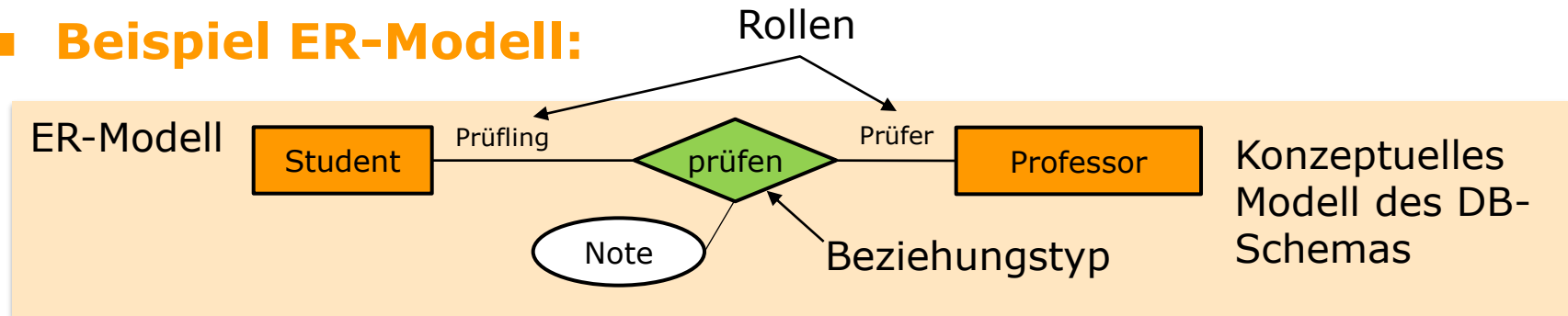


■ Semantik: Ein Beziehungstyp beschreibt die Menge aller Beziehungen, die zwischen zwei (oder mehr) Entitätsmengen vorherrschen; dementsprechend wird ein Beziehungstyp zwischen zwei (oder mehr) Entitätstypen definiert; Beziehungen können auch Attribute besitzen → Beziehungsattribute

- Ist ein Entitätstyp mehrfach an einem Beziehungstyp beteiligt, können Rollennamen vergeben werden; diese sind bei binären Beziehungstypen optional

Beziehungstyp

■ Beispiel ER-Modell:



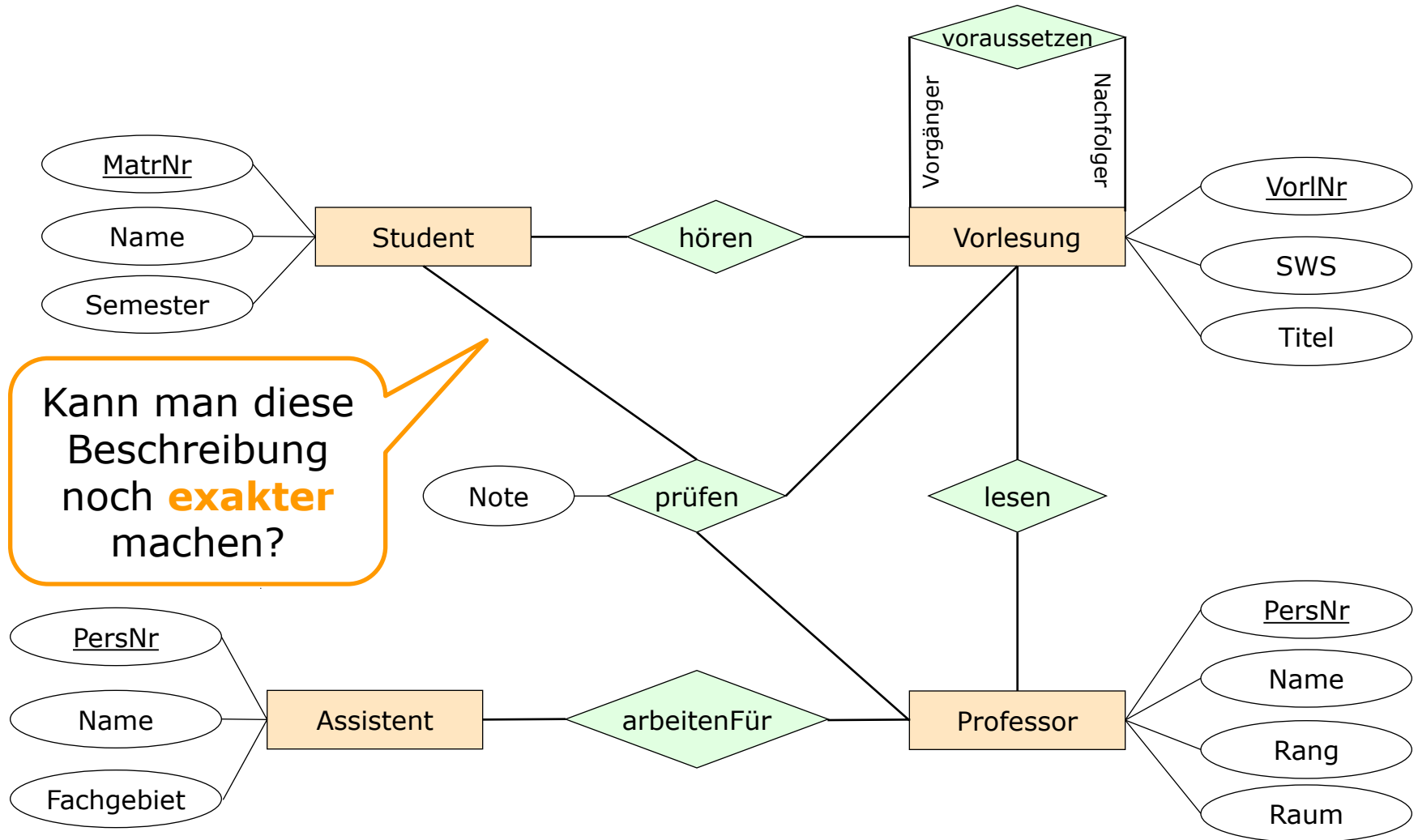
Modellierungsaufgabe



- Erstellen Sie ein **ER-Modell** für ein **Universitätsinformationssystem**
- Dabei soll folgender **Realitätsausschnitt** abgebildet werden:
 - Es sollen Studenten erfasst werden können. Für einen Studenten sollen seine Matrikelnummer, sein Name und sein aktuelles Semester erfasst werden.
 - Studenten besuchen Vorlesungen, die aufeinander aufbauen können, d.h., eine Vorlesung setzt u.U. den Besuch einer anderen Vorlesung voraus. Vorlesungen sind gekennzeichnet durch eine Vorlesungsnummer, einen Titel sowie die Anzahl der Semesterwochenstunden.
 - Vorlesungen werden von Professoren abgehalten. Ein Professor ist durch seine Personalnummer, seinen Namen, seinen Rang und sein Büro gekennzeichnet. Professoren halten Prüfungen ab – d.h., Studenten werden durch Professoren beurteilt.
 - Darüber hinaus führt ein Professor Forschungsarbeiten aus, wobei er durch Assistenten unterstützt sein kann. Diese sind durch ihre Personalnummer, ihren Namen sowie ein Fachgebiet gekennzeichnet.
 - ...

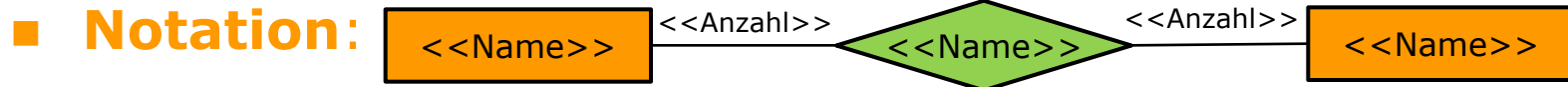
Entity-Relationship-Modell

Beispiel: Universitätsschema (Chen-Notation)



Kardinalitäten

■ ER-Konzept: Kardinalitäten



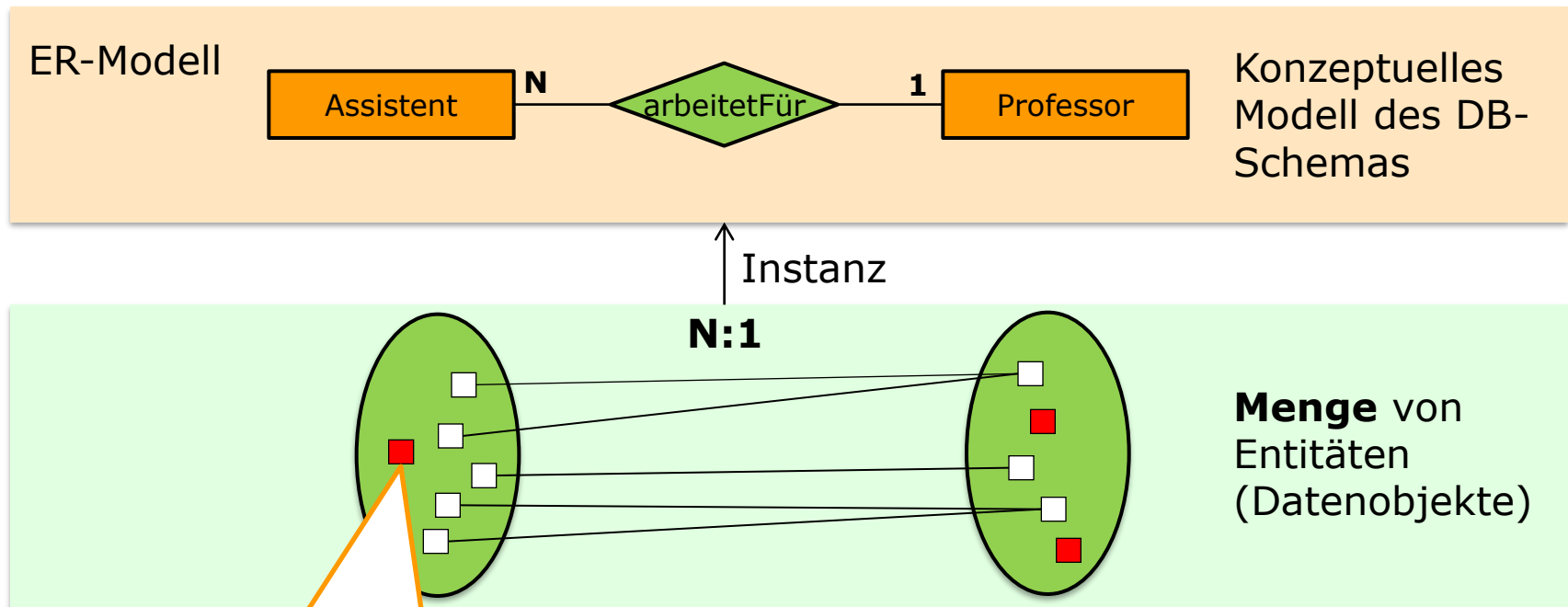
■ **Semantik:** Eine Kardinalität charakterisiert eine Beziehung exakter durch die Angabe, wie viele Entitäten der einen Entitätsmenge **mit wie vielen Entitäten** der anderen Entitätsmenge **maximal in Beziehung** stehen können

■ **Formen:**

- 1:1
- 1:N
- N:1
- N:M

Kardinalitäten

■ Beispiel ER-Modell:

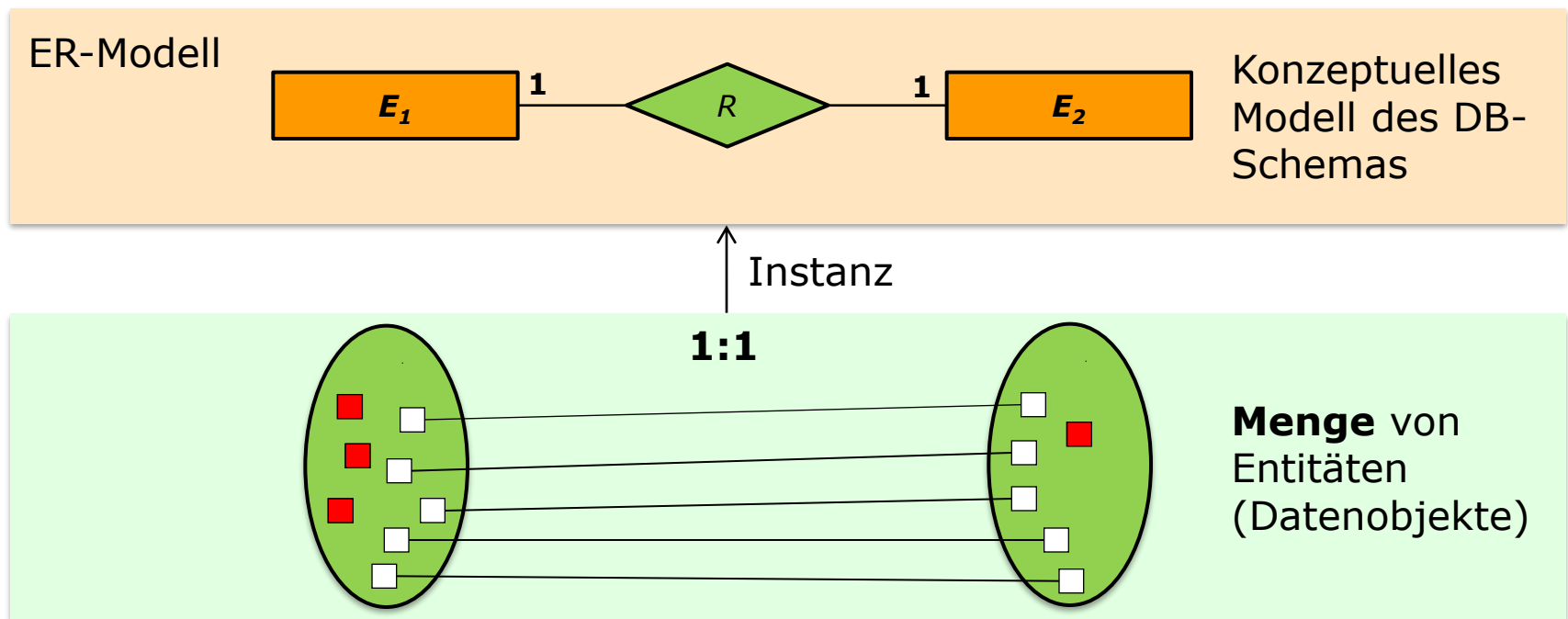


Es darf aber nach wie vor Entitäten geben, die mit keiner anderen Entität in Beziehung stehen!

Kardinalitäten

1:1-Beziehungen

- Jedem Entity e_1 vom Entitätstyp E_1 ist maximal ein Entity e_2 aus E_2 zugeordnet und umgekehrt
- **Beispiele:** Mann *ist verheiratet mit* Frau, Prospekt *beschreibt* Produkt

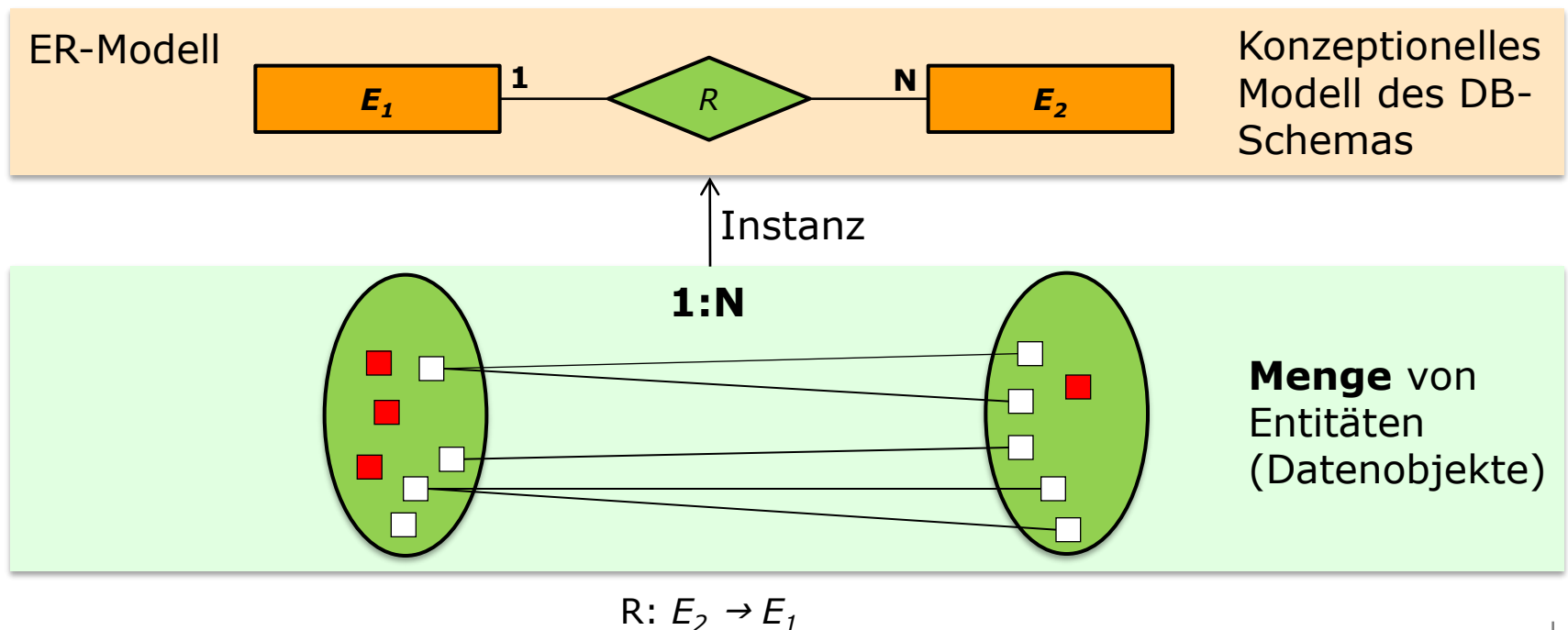


$$R: E_1 \rightarrow E_2 \text{ bzw. } R^{-1}: E_2 \rightarrow E_1$$

Kardinalitäten

1:N-Beziehungen

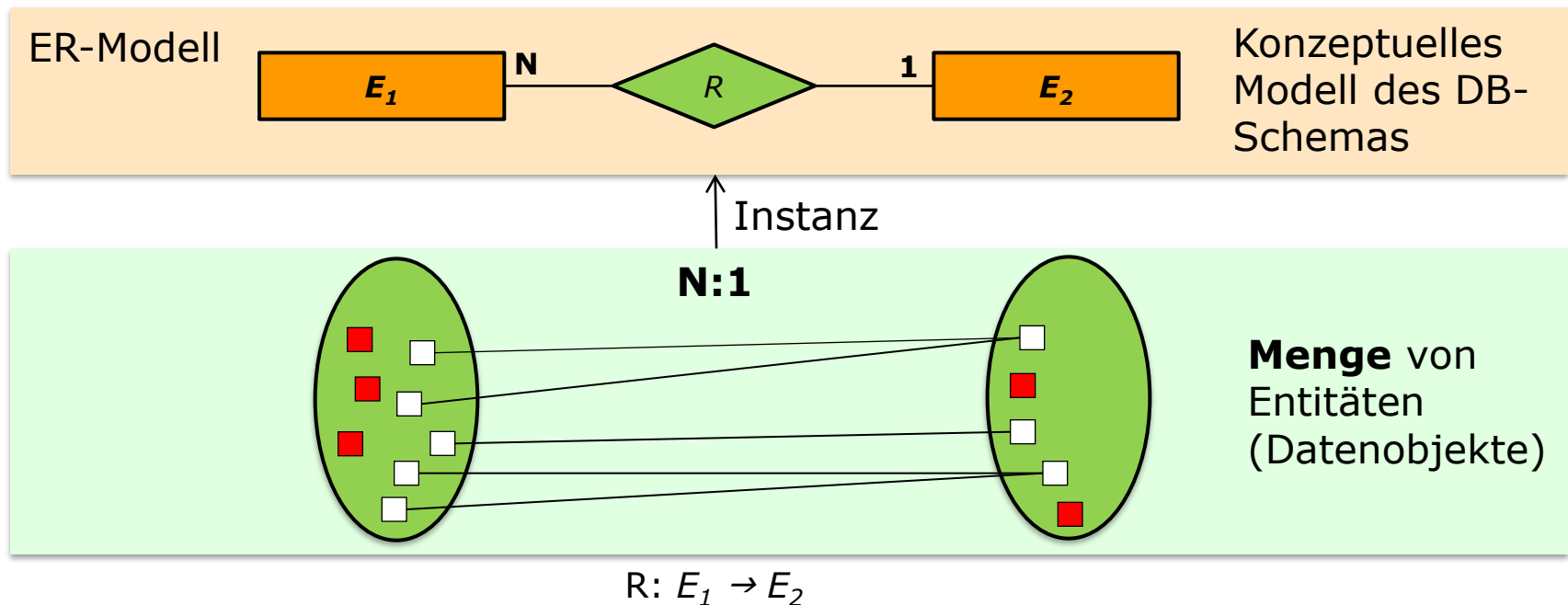
- Jeder Entität e_1 vom Entitätstyp E_1 sind beliebig viele Entitäten E_2 zugeordnet, aber zu jeder Entität e_2 gibt es maximal ein e_1 aus E_1
- **Beispiele:** Professor *liest* Vorlesung, Mutter *hat* Kind, Lieferant *liefert* Produkt



Kardinalitäten

N:1-Beziehungen

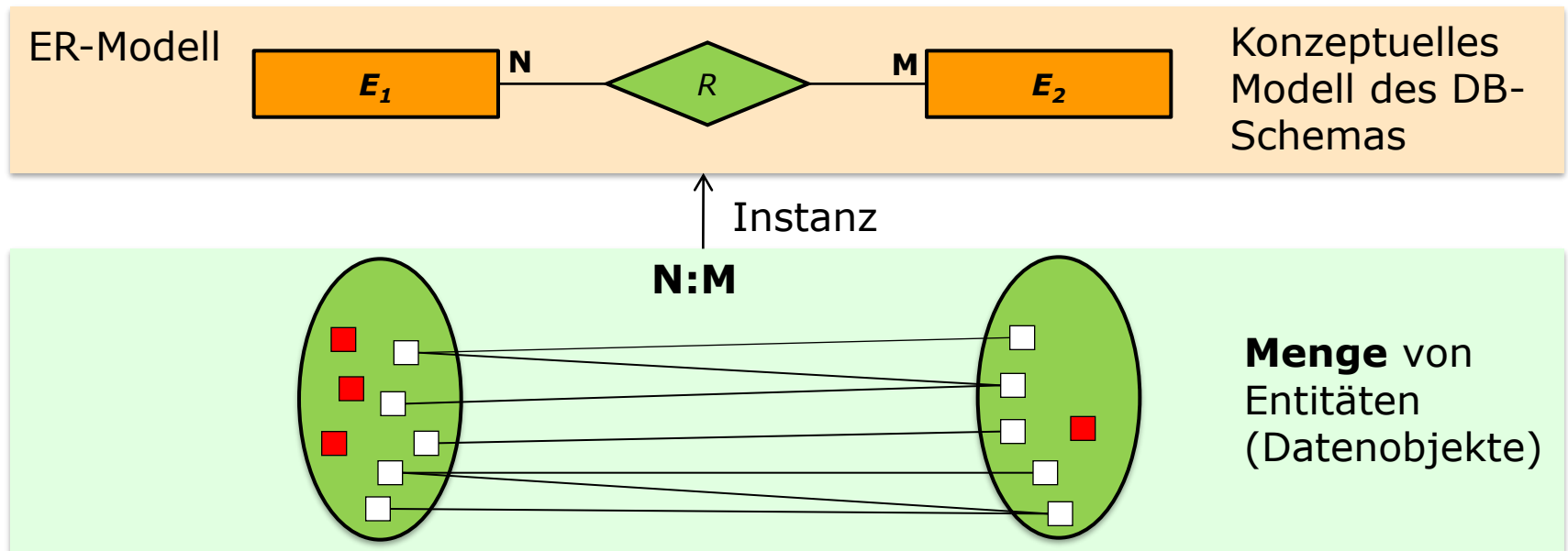
- Invers zu 1:N, d.h., jeder Entität e_1 vom Entitätstyp E_1 wird maximal eine Entität e_2 eines Entitätstyps E_2 zugeordnet
- **Beispiele:** Assistent *arbeitet für* Professor, Wein *wird produziert von* Erzeuger

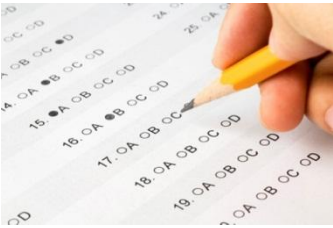


Kardinalitäten

M:N-Beziehungen

- Keine Restriktionen
- **Beispiele:** Student *hört* Vorlesung, Bestellung *umfasst* Produkte





Welche Aussage stimmt für korrekt modellierte Kardinalitäten von Relationstypen **nicht**?

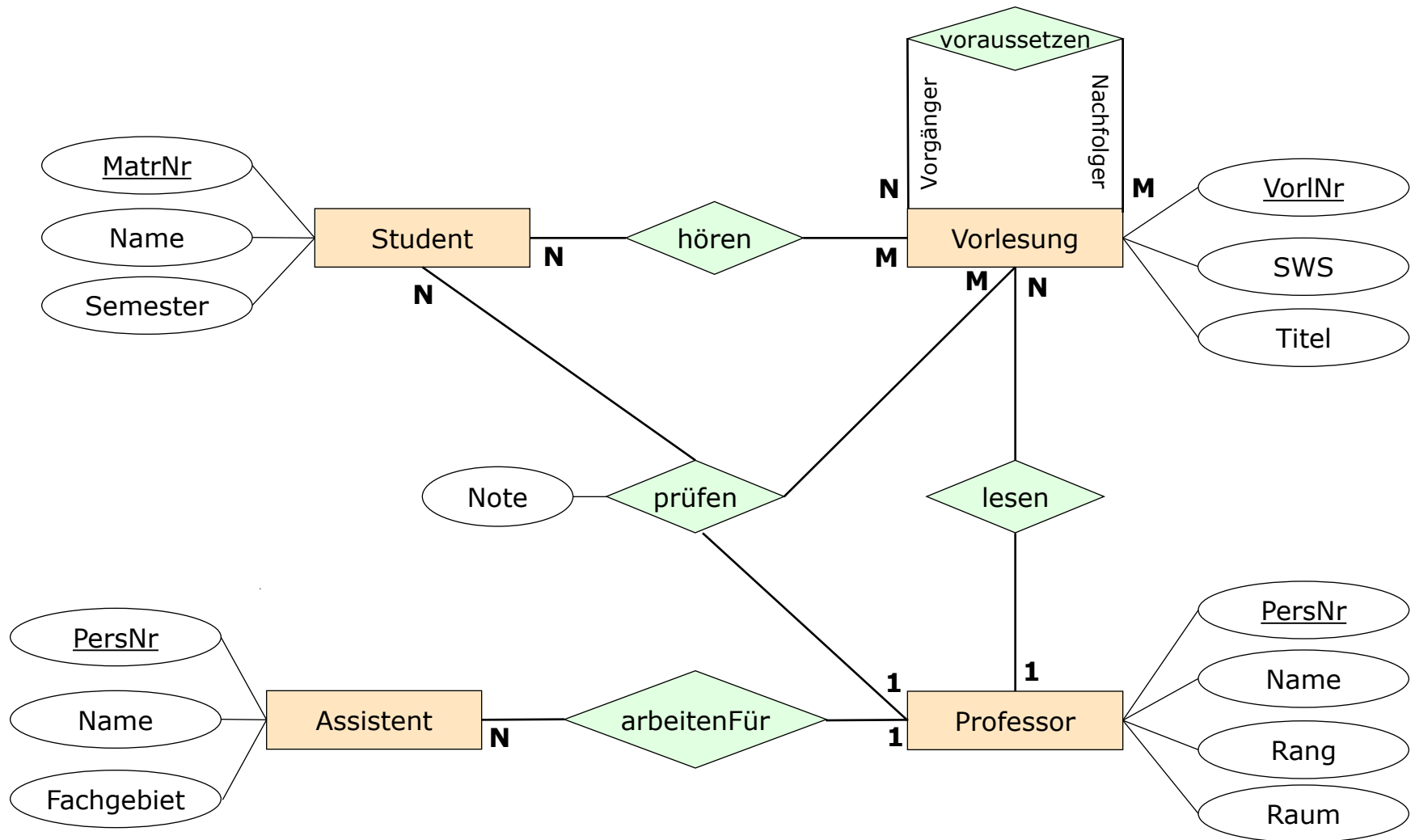
- ☐ Schwache Entitätstypen können eine N:M, 1:N und 1:1 Beziehung eingehen.
- ☒ Steht ein Entitätstyp mit sich selbst in Beziehung, gilt stets eine N:M Kardinalität.
- ☐ Kardinalitäten beschreiben die Anzahl an Beziehungen, die eine Entität eingehen kann.
- ☐ Die Kardinalität eines Beziehungstypen ist stets definiert.
- ☐ Kardinalitäten zeigen an, ob eine Beziehung verpflichtend oder optional ist.

Modellierungsaufgabe



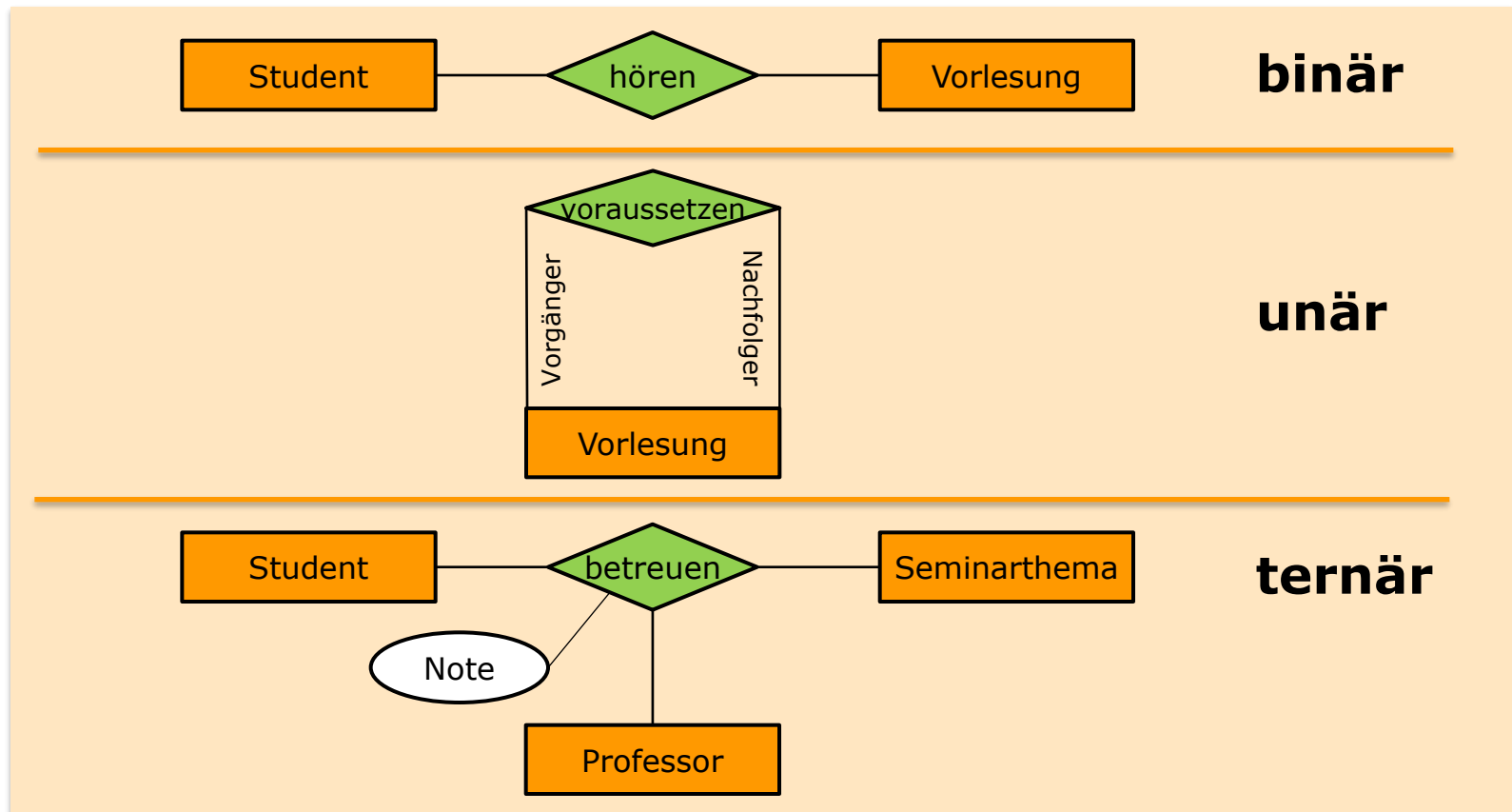
- Erweitern Sie das soeben erstellte ER-Modell für das **Universitätsinformationssystem** um Kardinalitäten.

Bsp.: Universitätsschema mit Kardinalitäten



Grad/Stelligkeit von Beziehungen

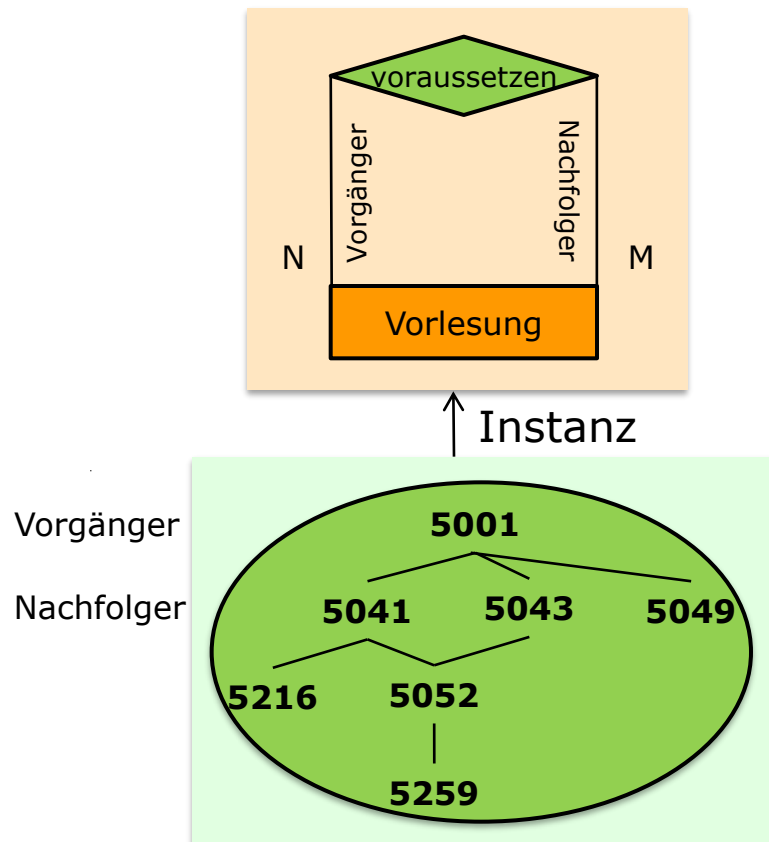
- Grad/Stelligkeit = **Anzahl** der beteiligten Entitätstypen
- Häufig: binär (unär und ternär möglich)



Kardinalitäten

bei einstelligen (unären) Beziehungen

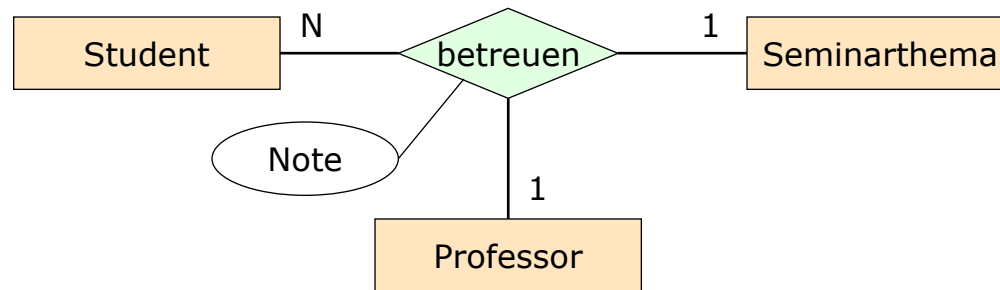
- **Rekursive Beziehungen** bilden eine **hierarchische Struktur** ab und werden als **unäre Beziehungen** dargestellt
- 1:1-, 1-N- und N:M-Beziehungen sind möglich



Kardinalitäten

bei ternären (mehrstelligen) Beziehungen

- Gegeben ist die ternäre Beziehung *betreuen* zwischen den Entitätstypen *Student*, *Professor*, *Seminarthema*.



betreuen: Professor x Student $\rightarrow$ Seminarthema

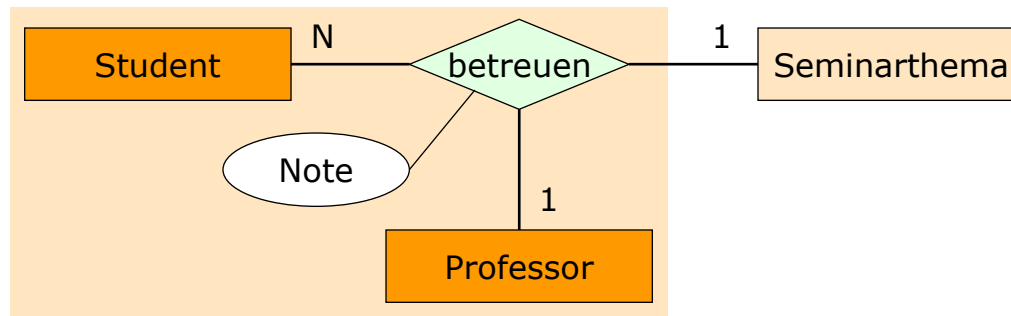
betreuen: Seminarthema x Student $\rightarrow$ Professor

- Welche Integritätsbedingungen werden durch diese Kardinalitäten festgelegt?

Ternäre Beziehung 1/3

Erläuterung

- betreuen: Student x Professor → Seminarthema

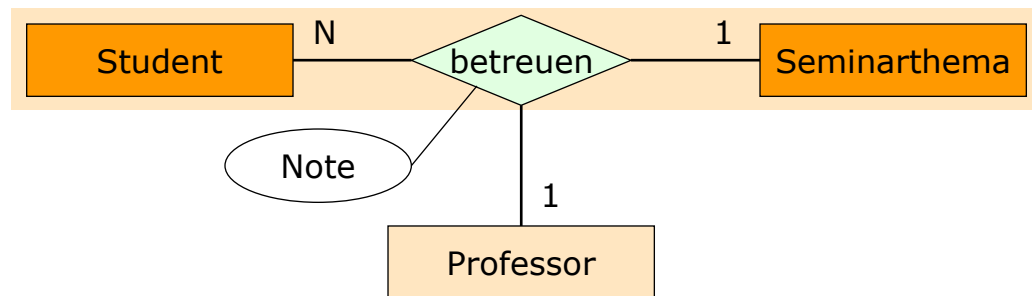


- Ein Student darf bei demselben Professor bzw. derselben Professorin nur ein Seminarthema bearbeiten (damit ein breites Spektrum abgedeckt wird).

Ternäre Beziehung 2/3

Erläuterung

- betreuen: Student x Seminarthema → Professor

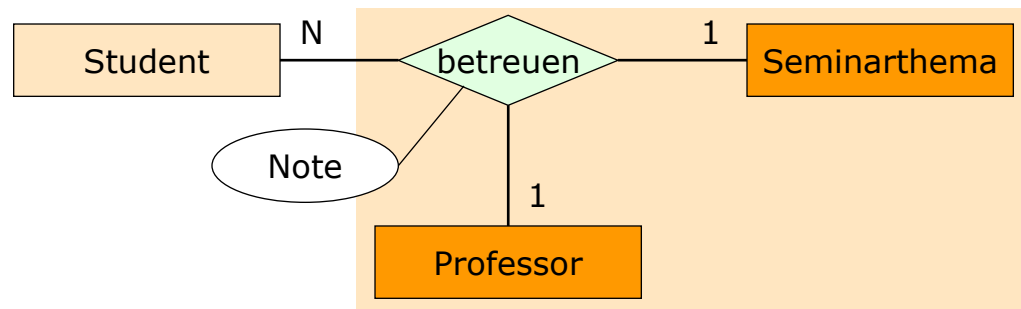


- Ein Student darf dasselbe Seminarthema nur bei einem Professor bzw. einer Professorin bearbeiten (= nicht wiederverwenden – sie dürfen also nicht bei anderen ProfessorInnen ein schon einmal erteiltes Seminarthema nochmals bearbeiten).

Ternäre Beziehung 3/3

Erläuterung

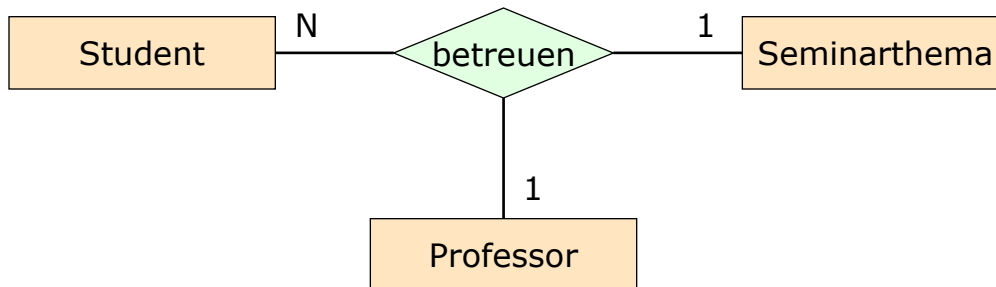
- betreuen: Professor x Seminarthema → Student



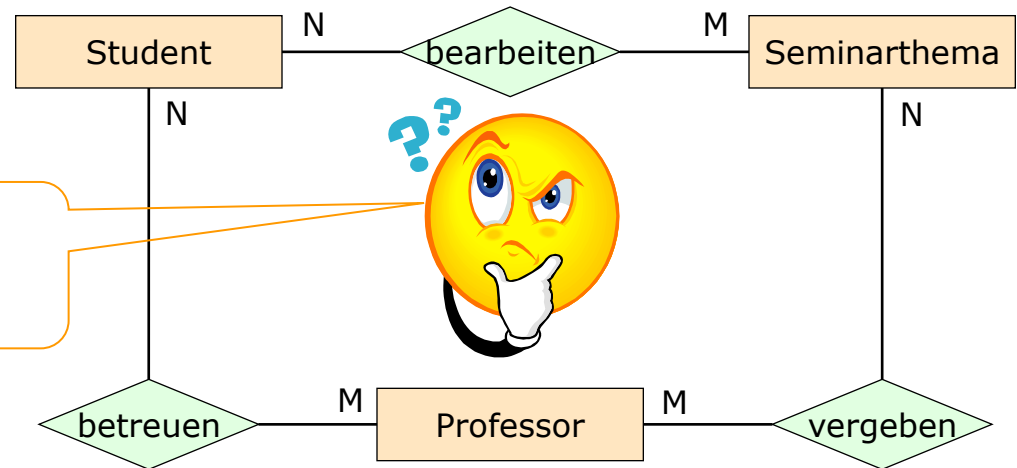
- Ein Professor kann dasselbe Seminarthema an mehrere Studenten vergeben (= „wiederverwenden“).

Ternäre versus binäre Beziehung 1/4

Ternäre Beziehung:

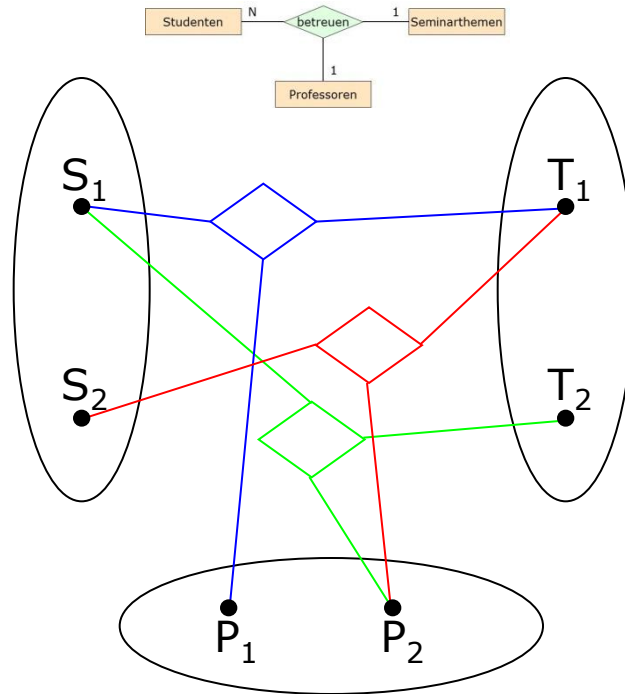


Ist eine ternäre Beziehung immer durch eine semantisch äquivalente Menge von binären Beziehungen ersetzbar?



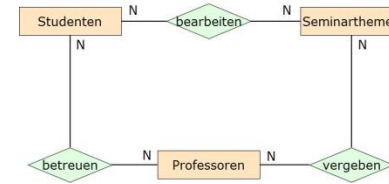
Ternäre versus binäre Beziehung 2/4

Ausprägung am Beispiel



Daten der ternären Beziehung:
betreuen

Student	Seminarthema	Professor
S ₁	T ₁	P ₁
S ₁	T ₂	P ₂
S ₂	T ₁	P ₂



Daten der drei binären Beziehungen:
bearbeiten betreuen vergeben

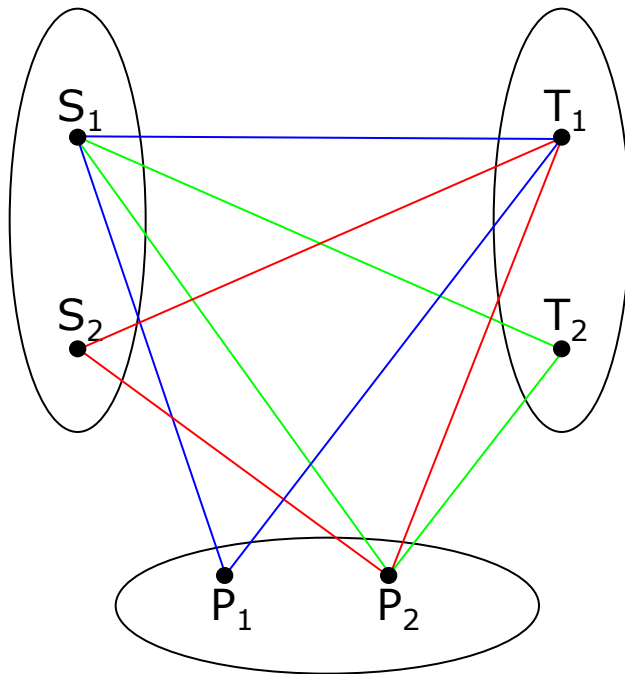
Student	Seminarthema
S ₁	T ₁
S ₁	T ₂
S ₂	T ₁

Student	Professor
S ₁	P ₁
S ₁	P ₂
S ₂	P ₂

Seminarthema	Professor
T ₁	P ₁
T ₂	P ₂
T ₁	P ₂

Ternäre versus binäre Beziehung 3/4

Rekonstruktion der Beziehungen



Student	Seminarthema	Professor
S_1	T_1	P_1
S_1	T_2	P_2
S_2	T_1	P_2

aber auch: $S_1 - T_1 - P_2$!

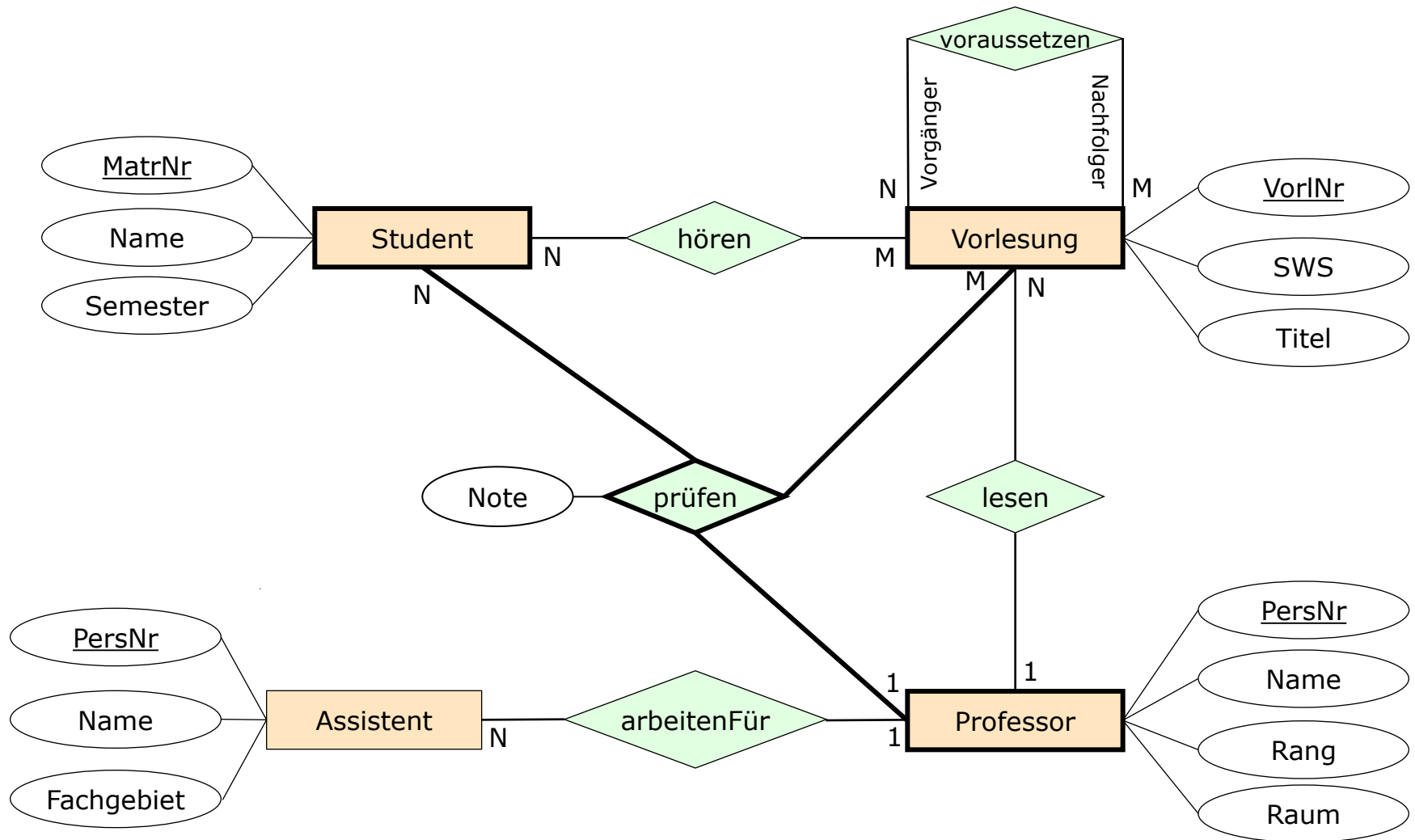
- ursprüngliche Beziehungsausprägung kann nicht mehr rekonstruiert werden
- Antwort: Mehrstellige Beziehungen können nicht direkt durch mehrere zweistellige Beziehungen ersetzt werden.

Ternäre versus binäre Beziehung 4/4

Conlusio

- Die ternäre Beziehung **betreuen** ist nicht äquivalent mit den drei binären Beziehungen **betreuen**, **vergeben** und **bearbeiten**!
- Eine Instanz (**Student**, **Professor**, **Seminarthema**) der Beziehung **betreuen** bedeutet, dass es auch folgende Instanzen gibt:
 - **betreuen**: (**Student**, **Professor**)
 - **bearbeiten**: (**Student**, **Seminarthema**)
 - **vergeben**: (**Professor**, **Seminarthema**)
- Die Umkehrung muss nicht gelten, es können drei Instanzen der binären Beziehungen existieren, die keine ternäre Entsprechung haben.

Bsp.: Universitätsschema mit Kardinalitäten



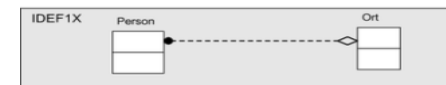
Erweiterungen des ER-Modells

■ Ziel

- Erfassung von **zusätzlicher Semantik** aus dem **Realitätsausschnitt** durch das ER-Modell
- Erhöhung der **Modellierungsgenauigkeit**

■ Erweiterungen

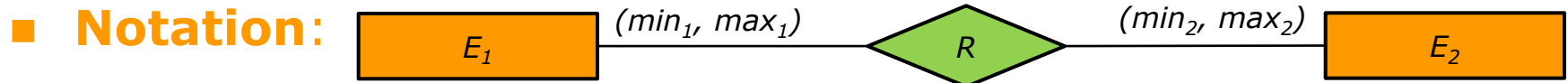
- (min,max)-Notation
- Existenzabhängige Entity-Typen
- Generalisierung und Spezialisierung
- Aggregation
- Abgeleitet, zusammengesetzte und mehrwertige Attribute
- ...



Problem: Erweiterungen sind nicht standardisiert!

(min, max)-Notation

■ EER-Konzept: (min, max)-Notation



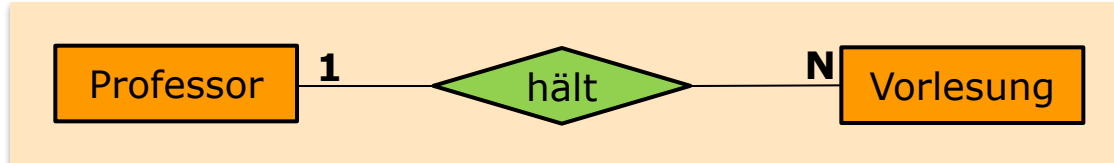
■ **Semantik:** Schränkt die möglichen Teilnahmen von Instanzen der beteiligten Entitätstypen an der **Beziehung** ein, indem ein minimaler und ein maximaler Wert vorgegeben wird

■ **Sonderfälle:**

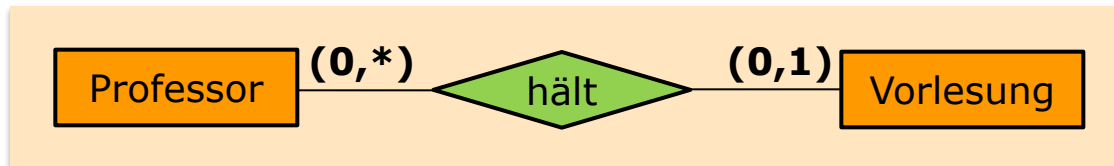
- Wenn es Entities geben darf, die gar nicht an der Beziehung "teilnehmen", so ist die min-Angabe = 0
- Wenn ein Entity beliebig oft "teilnehmen" darf, wird die max-Angabe durch * ersetzt
- [0..*] legt somit keine Einschränkung fest

Kardinalität versus (min, max)-Notation

- **Kardinalität:** Werte geben an, wie viele Entitäten eines Entitätstyps mit genau einer Entität des anderen am Beziehungstyp beteiligten Entitätstyps (und umgekehrt) in Beziehung stehen können oder müssen



- **(min,max)-Notation:** Werte geben an, an wie vielen Beziehungsausprägungen die Entitätsausprägungen mindestens teilnehmen müssen und an wie vielen sie höchstens teilnehmen dürfen



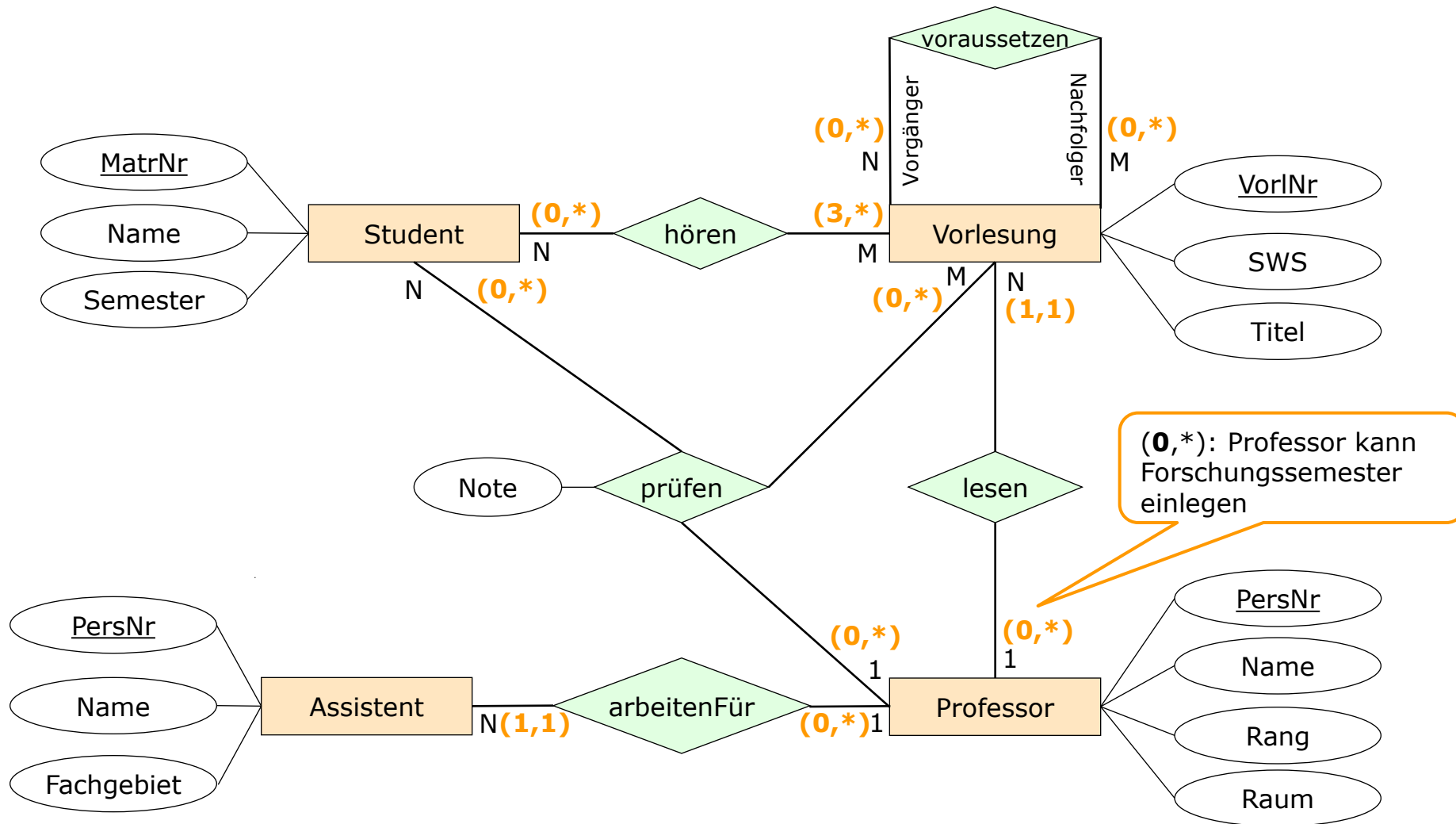
- Die (min,max)-Notation zählt die Ausprägungen von Beziehungen, während die Kardinalität Ausprägungen von Entitätstypen zählt

Modellierungsaufgabe



- Erweitern Sie das soeben erstellte ER-Modell für das **Universitätsinformationssystem** um **(min,max)-Kardinalitäten**.

Bsp.: (min, max)-Notation



Existenzabhängige Entitätstypen 1/2

Schwache Entities

- **EER-Konzept:** Existenzabhängige Entitätstypen



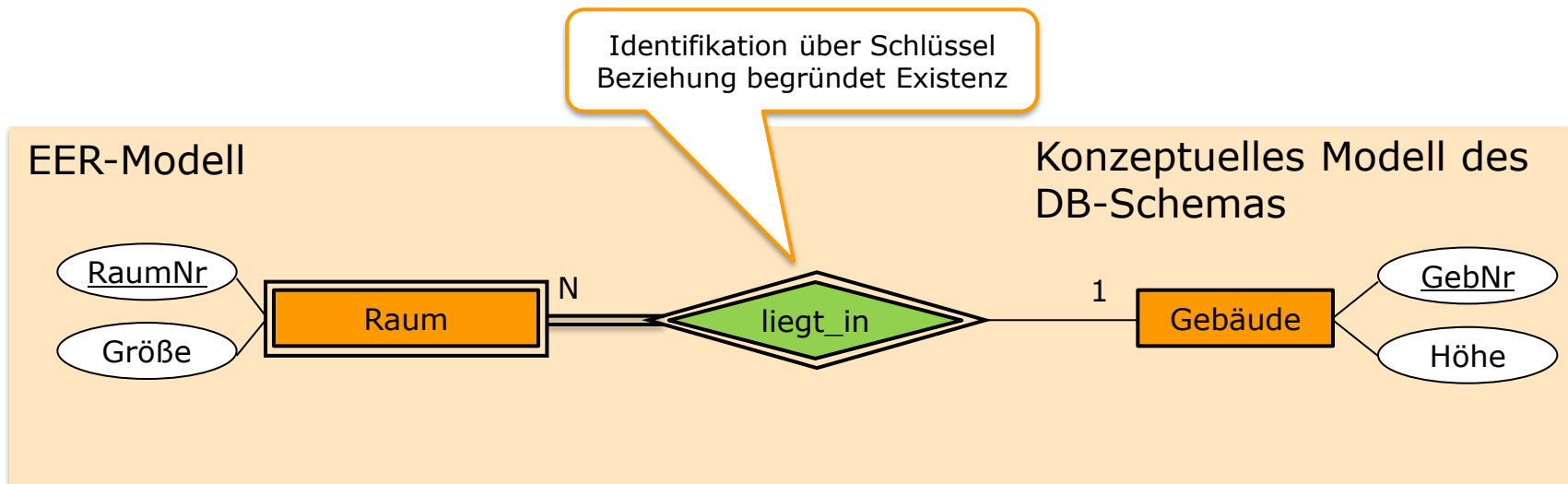
- **Semantik:** Existenzabhängige Entitätstypen (schwache Entities) sind
 - in ihrer Existenz von einem anderen, übergeordneten Entity **abhängig** (= starkes Entity)
 - oft nur in **Kombination** mit dem Schlüssel des übergeordneten Entity **eindeutig identifizierbar** (= Identifikation über funktionale Beziehung)
- Schwache Entitäten stehen in der Regel in einer **N:1**-Beziehung zu ihrer übergeordneten Entität, selten in einer **1:1**-Beziehung

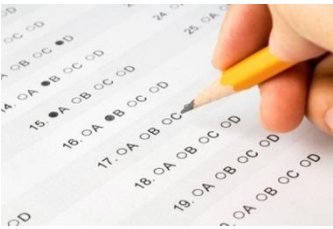
Existenzabhängige Entitätstypen 2/2

Schwache Entities

■ Beispiel EER-Modell:

- Ist die Nummer eines Raumes nur innerhalb eines Gebäudes eindeutig, so ist der Schlüssel von Räume eine Kombination aus Raum- und Gebäudenummer





Welche Aussagen über schwache Entitätstypen sind wahr?

- ✓ ☒ Bei schwachen Entitätstypen ist die Beziehung zum starken Entitätstyp immer identifizierend.
- ☐ Bei schwachen Entitätstypen ist die Beziehung zum starken Entitätstyp nie identifizierend.
- ✓ ☒ Existenzabhängige (schwache) Entitätsmengen hängen in ihrer Existenz von einer übergeordneten (starken) Entitätsmenge ab.
- ☐ Jede nicht identifizierende Beziehung erzeugt einen schwachen Entitätstyp.
- ✓ ☒ Jeder schwache Entitätstyp hat einen zusammengesetzten Primärschlüssel.

Aggregation 1/2

Teil-von-Beziehung

Auf ein eigenes Symbol für diesen Beziehungstyp wurde verzichtet, da die Beziehung als 1:N-Beziehung für die spätere Verwendung in SQL hinreichend genau modelliert ist

■ EER-Konzept: Aggregation (Teil-von-Beziehung)

■ Notation:



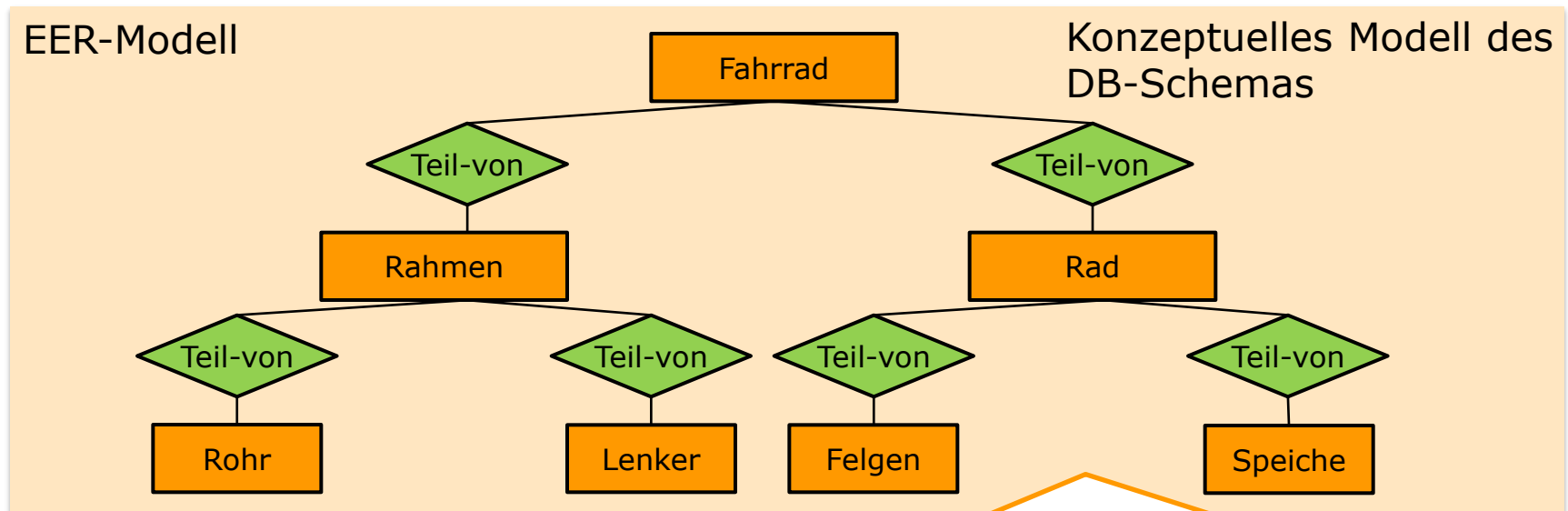
■ Semantik:

- Entitätstypen, die in ihrer Gesamtheit einen **komplexen Entitätstypen** bilden, werden einander zugeordnet
- Es gibt einen übergeordneten Entitätstyp ($\sim$ Ganze, Aggregat), dem ein oder mehrere Entitätstypen ($\sim$ Teile, Komponenten) untergeordnet sind
- Diese Art von Beziehung wird als **Besteht-aus** oder **Teil-von** (engl. part-of relationship) bezeichnet
- Ist ein **Spezialfall** einer **1:N-Beziehung**
- **Asymmetrische, transitive** Assoziation zwischen nicht gleichwertigen Partnern
 - **Asymmetrie:** B ist Teil von A, aber A ist nicht Teil von B
 - **Transitivität:** C ist Teil von B und B ist Teil von A, dann ist auch C Teil von A

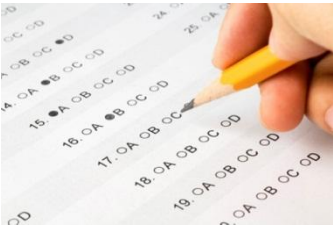
Aggregation 2/2

Teil-von-Beziehung

■ Beispiel EER-Modell:



Häufig, aber **nicht zwangsläufig**, ist ein **schwacher Entitätstyp** per **Aggregation** an einen **starken Entitätstyp** gekoppelt. Im Gegensatz zur Modellierung mittels schwacher Entities stellt eine **Aggregation keine existenzabhängige Beziehung** dar. Beispielsweise kann ein Motor auch ohne Auto, in das er eingebaut wird, existieren. Anders ist es bei Räumen. Ein Raum kann nur dann existieren, wenn es auch ein Gebäude gibt, das den Raum beinhaltet → Es besteht somit eine **starke konzeptionelle Nähe** zu **abhängigen Entitätstypen** (schwacher Entitätstyp)



Eine Aggregation ist ein Spezialfall einer 1:N-Beziehung mit eigener Semantik?

- ☒ stimmt
- ☐ stimmt nicht

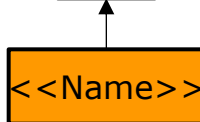
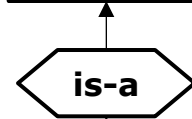


Generalisierung/Spezialisierung 1/3

is-a-Beziehung mit Vererbung von Struktur

■ EER-Konzept: Generalisierung/Spezialisierung

■ Notation: <<Name>> Obertyp



Obertyp

Untertyp

ACHTUNG:

- **Aggregation** drückt eine **"Teil-von"-Beziehung zwischen einzelnen Entitäten** aus, z.B.: "Motor ist Teil von Auto"
- **Generalisierung/Spezialisierung** hingegen drückt eine **Teilmengen-Beziehung zwischen Entitätsmengen** aus, z.B.: "Studenten sind eine Teilmenge von Personen"

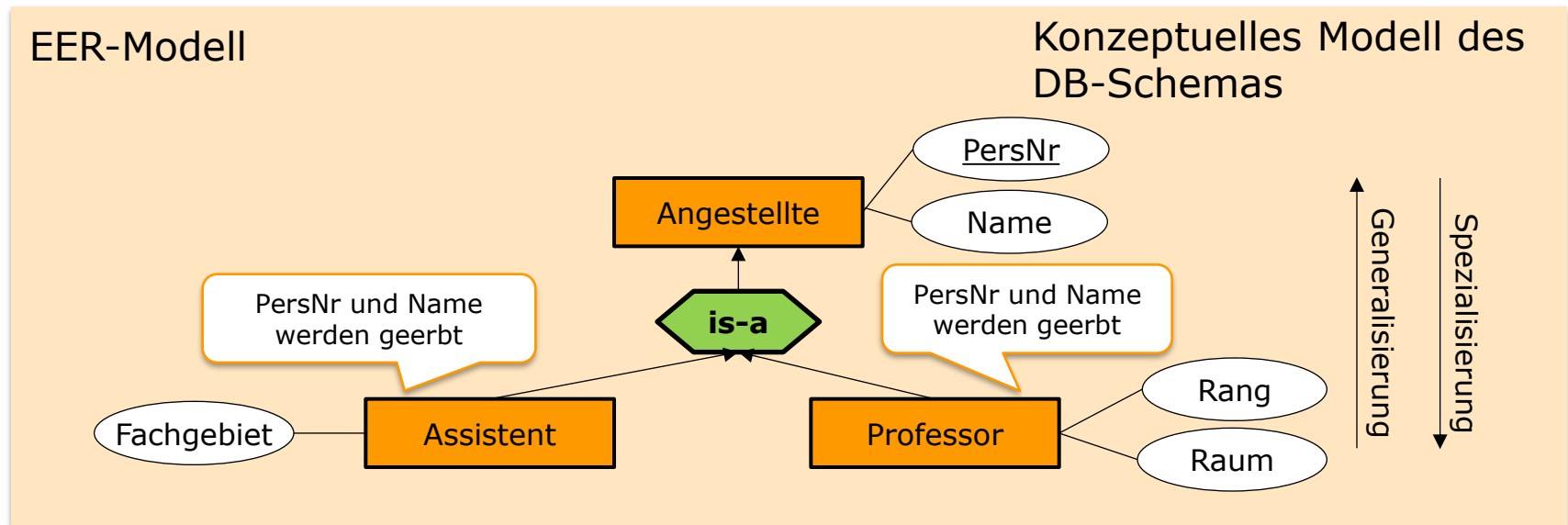
■ Semantik: <<Name>> Untertyp

- Drückt eine Teilmengenbeziehung zwischen den Entitätsmengen aus; **Untertyp erbt die Attribute und Beziehungen von Obertyp**; Ober- und Untertyp besitzen den **gleichen Primärschlüssel**
- Zwei Wege, um Teilmengenbeziehung zu erstellen
 - **Generalisierung**
 - Eigenschaften ähnlicher Entity-Typen werden „herausfaktoriert“ und einem gemeinsamen **Obertyp** zugeordnet
 - Der Obertyp wird durch diejenigen Attribute beschrieben, die den abgeleiteten Untertypen gemeinsam sind
 - **Spezialisierung**
 - **Untertypen** werden von Obertypen abgeleitet
 - Abgeleitete Untertypen haben neben den vererbten Attributen eigene Attribute

Generalisierung/Spezialisierung 2/3

is-a-Beziehung mit Vererbung von Struktur

■ Beispiel EER-Modell:

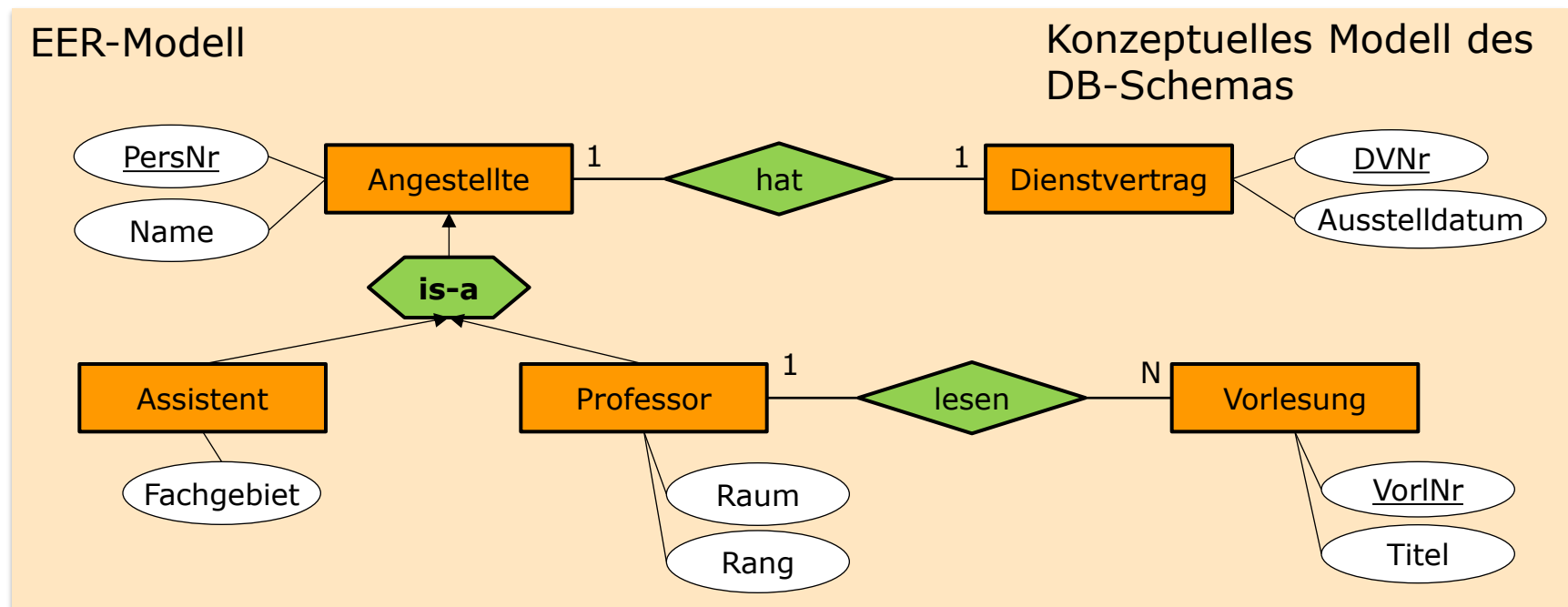


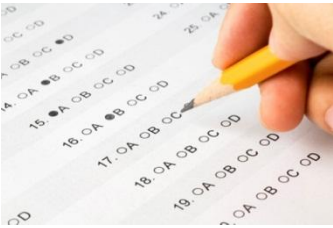
Generalisierung/Spezialisierung 3/3

is-a-Beziehung mit Vererbung von Struktur

■ Beispiel EER-Modell:

- Beziehung **hat** wird auf die Untertypen **Assistent** und **Professor** vererbt
- Beziehung **lesen** bezieht sich nur auf den Untertyp **Professor**





Welche Aussagen über das erweiterte ER-Modell sind wahr?

- ✓ ☒ Eine is-a-Beziehung ist eine Beziehung zwischen einem Obertyp und einem Untertyp, wobei der Untertyp alle Attribute des Obertyps erbt, die um eigene Attribute und Beziehungen ergänzt werden können.
- ☐ Unter einer Generalisierung versteht man den Prozess der Gewinnung von Untertypen aus einem gegebenen Obertyp.
- ✓ ☒ Unter einer Spezialisierung versteht man den Prozess der Gewinnung von Untertypen aus einem gegebenen Obertyp.
- ☐ Obertyp und Untertyp besitzen unterschiedliche Primärschlüssel.
- ✓ ☒ Beziehungen eines Obertyps werden auf die Untertypen vererbt.

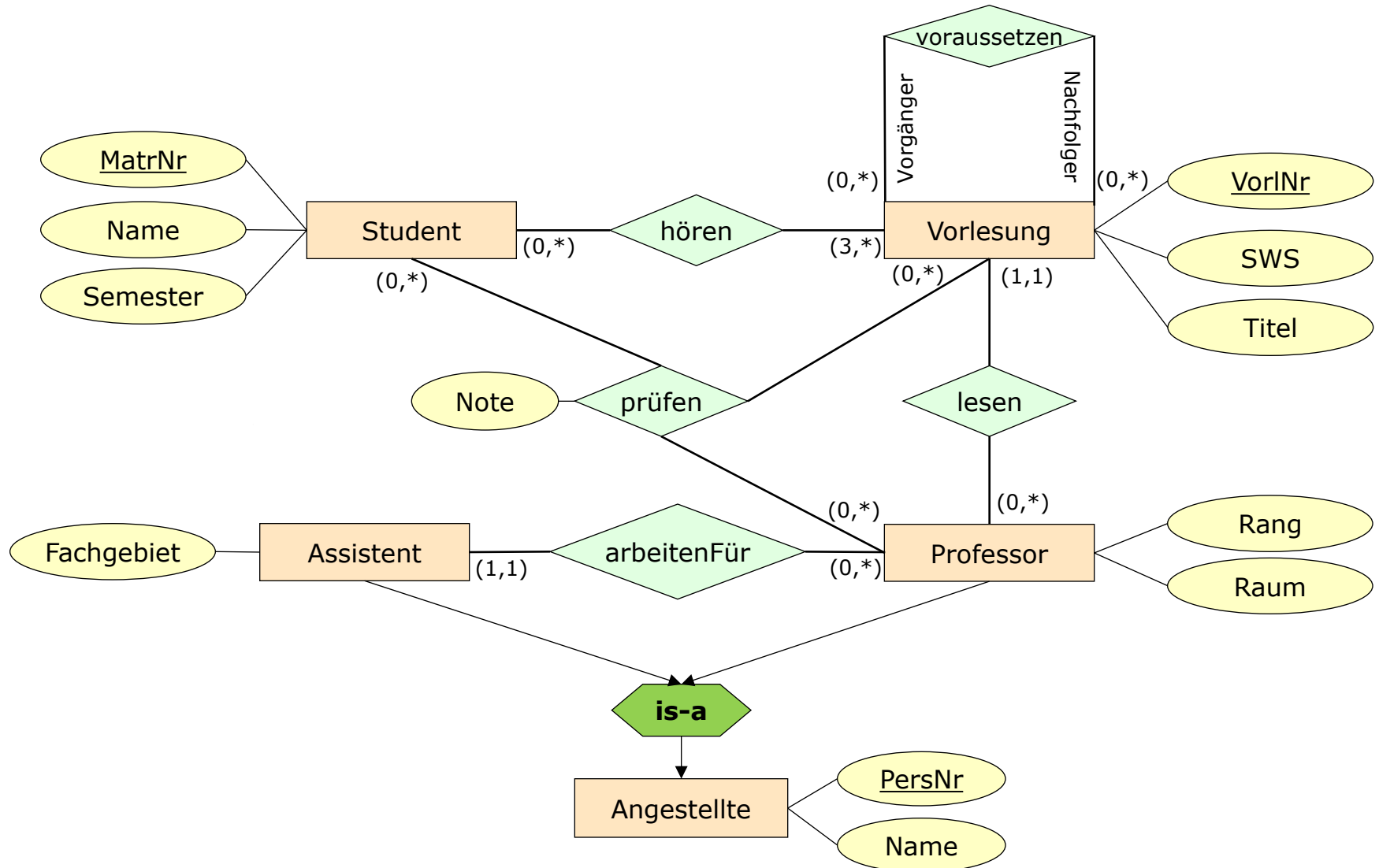
Modellierungsaufgabe



- Erweitern Sie das soeben erstellte ER-Modell für das **Universitätsinformationssystem** um Vererbungsbeziehungen, wo sinnvoll.

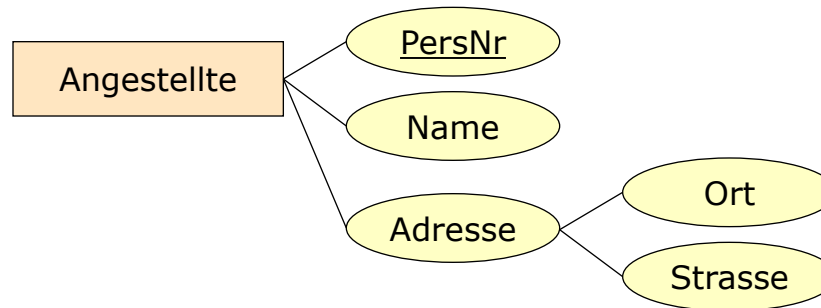
Bsp.: Universitätsschema

mit (min,max)-Notation und Generalisierung



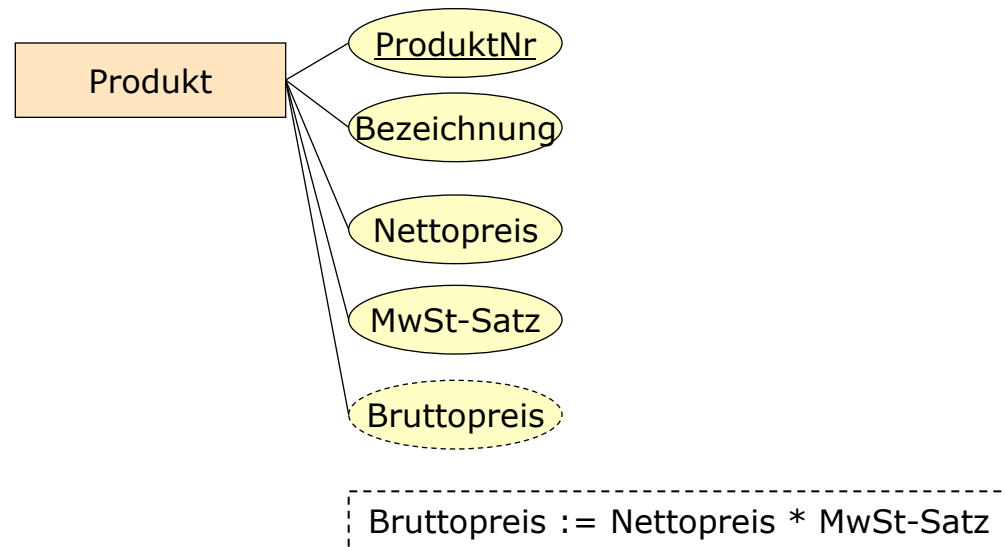
Strukturierte Attribute

- Attributgruppen, die Gemeinsamkeiten hinsichtlich ihrer Bedeutung und ihrer Verwendung haben



Abgeleitete Attribute

- Attribute, deren Werte nicht abgespeichert werden, sondern durch eine Anfrage an die Datenbank bestimmt werden können.



Überblick

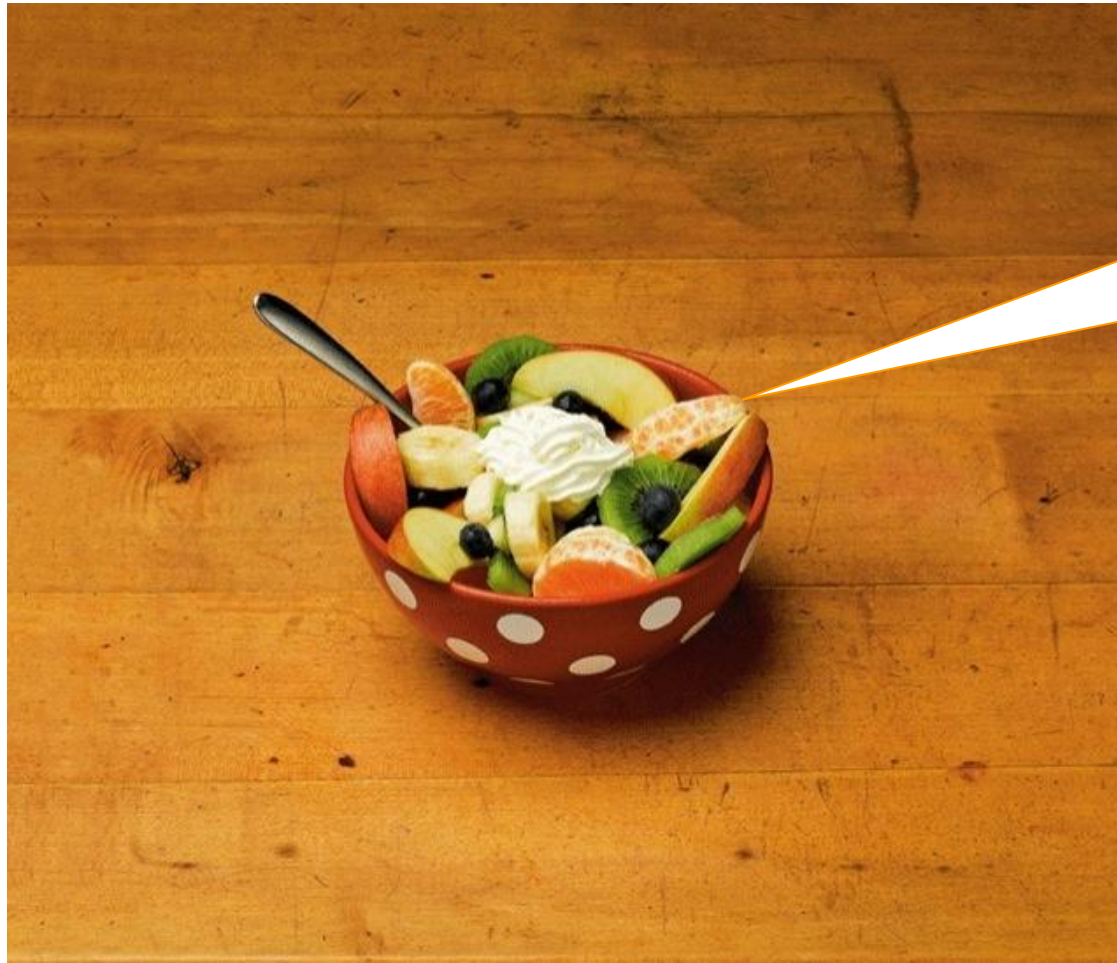
- Datenbankentwurfsschritte
- Modellbegriff
- Entity-Relationship-Modell
- Erweiterungen des Entity-Relationship-Modells
- Vorgehen
- Designprinzipien
- Anhang I: Definitionen

Vorgehen

- Nun haben wir die **Modellierungssprache** (E)ER mit seiner **Syntax** und **Semantik** kennengelernt
- **Aber:**
 - Wie kann man bei der Modellierung nun **systematisch vorgehen**?
 - D.h., wie kommt man von der **Realwelt** zu einem **konzeptuellen Modell**?

"Abstraktion"

oder "Wie kommt man von der Realwelt zu einem konzeptuellen Modell?"



Zu **modellierender Sachverhalt** der Realwelt: Obstsalat

Obstsalat

Kalorien

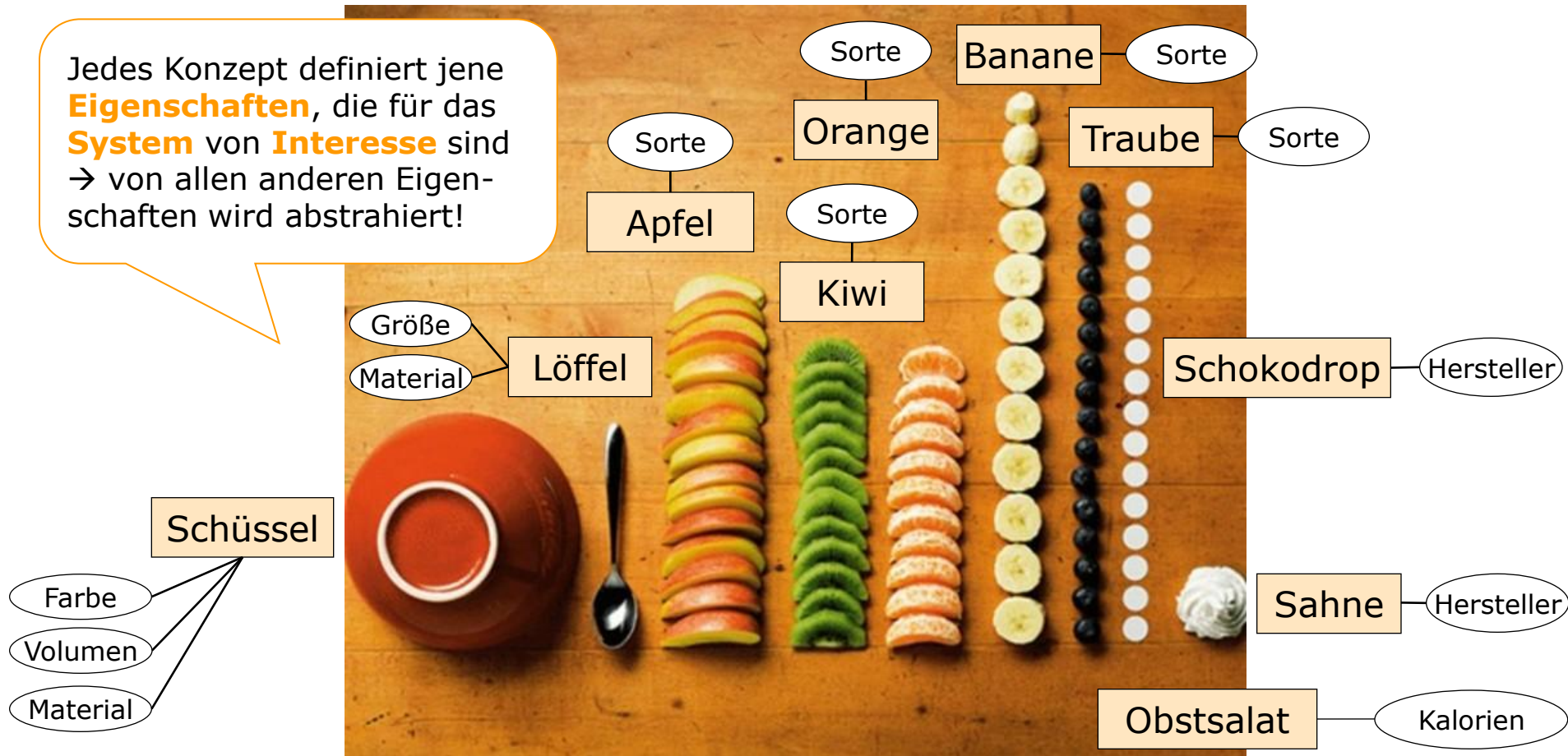
Ursus Wehrli: Die Kunst, auf(zu)räumen, Verlag KEIN & ABER, 2013

"Abstraktion"

oder "Wie kommt man von der Realwelt zu einem konzeptuellen Modell?"

■ Schritt 1: Klassifikation – Identifikation von Konzepten

Jedes Konzept definiert jene **Eigenschaften**, die für das **System** von **Interesse** sind
→ von allen anderen Eigenschaften wird abstrahiert!



Ursus Wehrli: Die Kunst, auf(zu)räumen, Verlag KEIN & ABER, 2013

"Abstraktion"

oder "Wie kommt man von der Realwelt zu einem konzeptuellen Modell?"

- **Schritt 1:** Klassifikation – Identifikation von **Konzepten** – **Entitätstypen/Attribute**
- **Gleichartige Objekte** werden klassifiziert – d.h. Objekte (Entitäten, Instanzen) mit **gemeinsamen Eigenschaften** (Attributen) werden zu einer Entitätsmenge bzw. einem Entitätstyp zusammengefasst
 - Entitäten eines Entitätstyps unterliegen gleicher Struktur (Attribute) und gleichen Integritätsbedingungen
 - Mathematisch: **Mengenbildung** (d.h. es dürfen keine Duplikate vorkommen!)

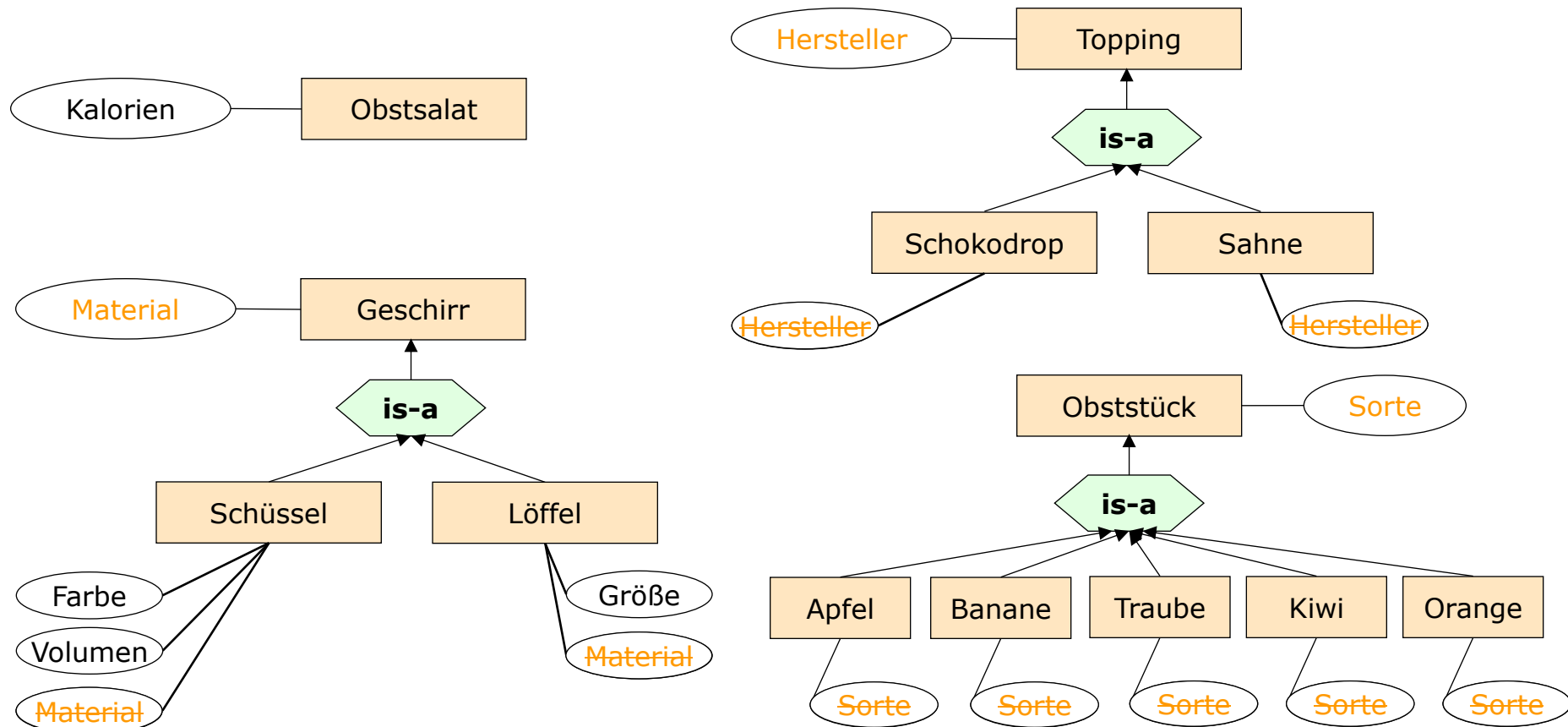
Artikel zum Thema Abstraktion:

Jeff Kramer. 2007. Is abstraction the key to computing?. Commun. ACM 50, 4 (April 2007), 36-42.

"Abstraktion"

oder "Wie kommt man von der Realwelt zu einem konzeptuellen Modell?"

- **Schritt 2:** Identifikation von **Beziehungen** zwischen Konzepten – **Verallgemeinerung/Generalisierung**



"Abstraktion"

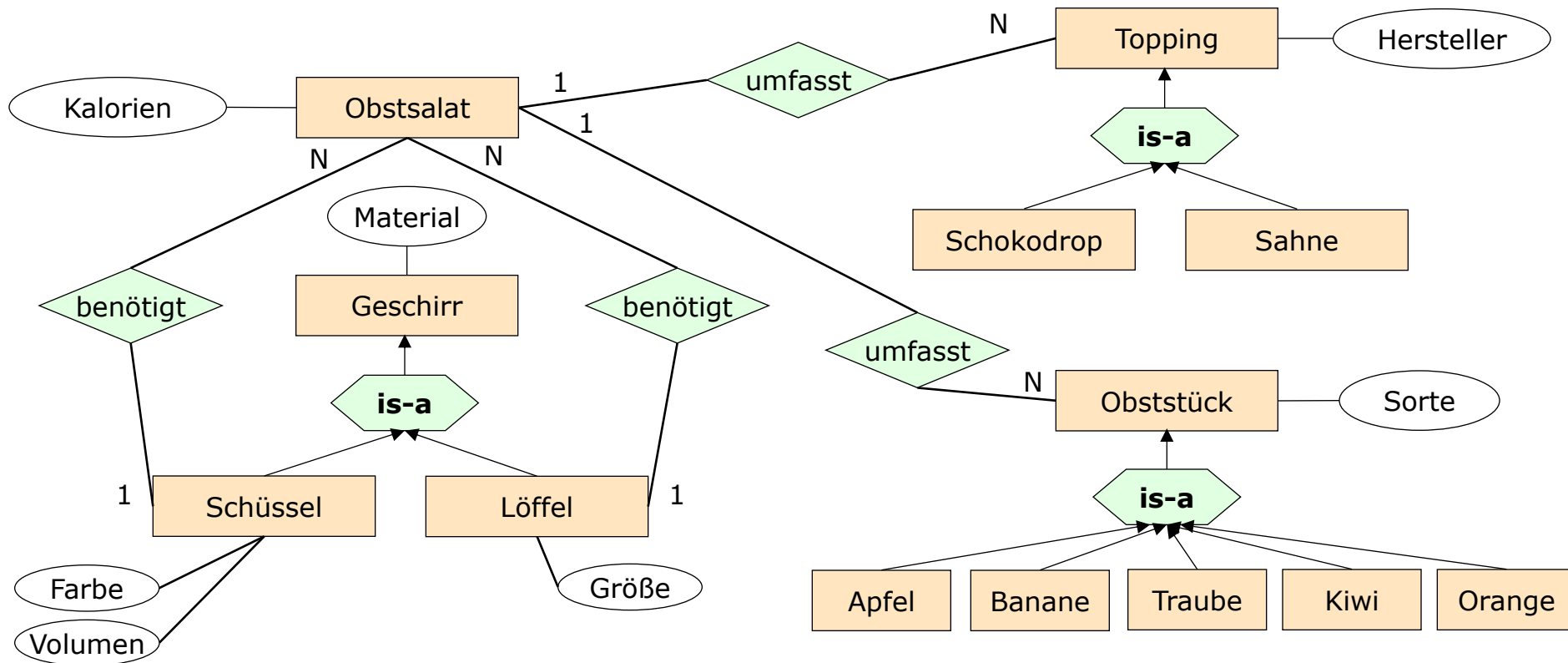
oder "Wie kommt man von der Realwelt zu einem konzeptuellen Modell?"

- **Schritt 2:** Identifikation von **Beziehungen** zwischen Konzepten – **Verallgemeinerung/Generalisierung**
- **Teilmengenbeziehungen** zwischen Elementen verschiedener Klassen
 - mathematisch: Bildung von Potenzmengen (bzw. **Teilmengen**)
 - wesentlich: **Vererbung** von Eigenschaften an Teilmengen

"Abstraktion"

oder "Wie kommt man von der Realwelt zu einem konzeptuellen Modell?"

■ Schritt 3: Identifikation von Beziehungen zwischen Konzepten – Aggregation



"Abstraktion"

oder "Wie kommt man von der Realwelt zu einem konzeptuellen Modell?"

- **Schritt 3:** Identifikation von Beziehungen zwischen Konzepten – **Aggregation**
- Zusammenfassung potentiell unterschiedlicher Teilobjekte (Komponenten) zu neuem Objekt
 - mathematisch: Bildung von **kartesischen Produkten**

"Abstraktion"

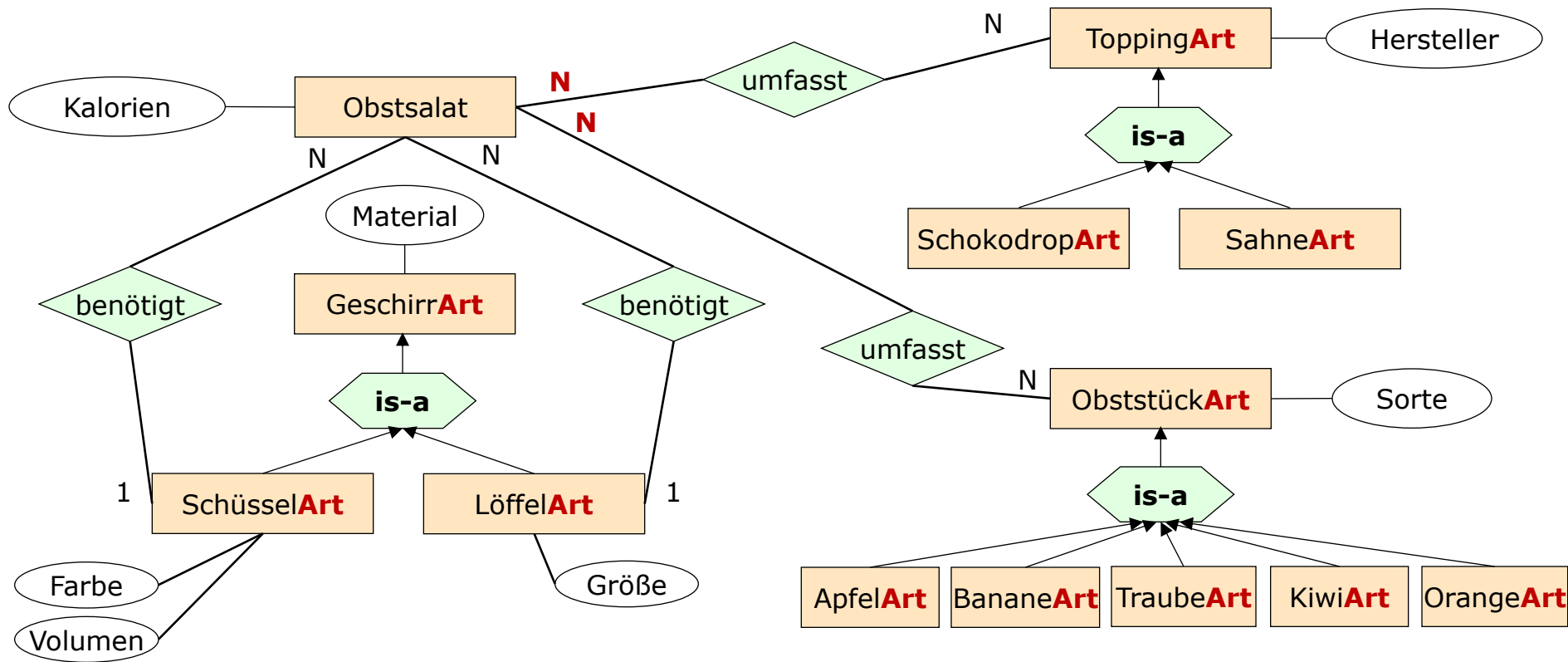
oder "Wie kommt man von der Realwelt zu einem konzeptuellen Modell?"

- In dem Modell der vorigen Folie ist **jedes individuelle Stück Obst** (bzw. auch Topping und Geschirr) **repräsentierbar**
 - bedingt sehr viele Instanzen (Speicherverbrauch!)
 - eventuell ist es aber für das System ausreichend zu wissen, welche Obstart (bzw. Toppingart bzw. Geschirrart) im Obstsalat verwendet wurde

"Abstraktion"

oder "Wie kommt man von der Realwelt zu einem konzeptuellen Modell?"

■ Alternativlösung



"Abstraktion"

oder "Wie kommt man von der Realwelt zu einem konzeptuellen Modell?"

■ Achtung

- Typischerweise gibt es nicht "die" eindeutige richtige Lösung
- Typischerweise gibt es verschiedene Modellierungsvarianten mit unterschiedlichen Eigenschaften (Vor-/Nachteilen je nach Situation) – hängt natürlich stark von den jeweiligen Systemanforderungen ab

■ Aber

- Es gibt bestimmte Qualitätsmerkmale für Modelle

Überblick

- Datenbankentwurfsschritte
- Modellbegriff
- Entity-Relationship-Modell
- Erweiterungen des Entity-Relationship-Modells
- Vorgehen
- Designprinzipien
- Anhang I: Definitionen

Qualitätsmerkmale für Modelle

■ Vollständigkeit

- Alle systemrelevanten Konzepte sollen abgebildet werden

■ Korrektheit

- Korrekte Abbildung der Konzepte und Beziehungen zwischen diesen wie in der Realwelt vorhanden, z.B. Kardinalitäten

■ Minimalität

- Nur systemrelevante Konzepte sollen abgebildet werden → sonst nichts

■ Lesbarkeit, Verständlichkeit

- Aussagekräftige Benennungen von Entitätstypen, Attributen, Beziehungen
- Verwendung entsprechender Klassifikationen von Konzepten, um Beziehungen auf höherem Abstraktionsgrad setzen zu können

Designprinzipien

Anwendungstreue

- Entitätstypen und Attribute sollen die Realität widerspiegeln
 - **Vorlesung** haben keine **Altersbeschränkung**
- Beziehungstypen sollen Verhältnisse der Realität widerspiegeln
 - **Student** und **Vorlesung** stehen in einer **M:N**-Beziehung
 - **N:1**, **1:N** oder **1:1** wären inkorrekte Wiedergaben der Realität
- Blick in die Zukunft
 - Welche Daten können entstehen?
 - **Tutorium** als **Freifach** im Lehrveranstaltungszeugnis
 - Außerordentliche Studierende
 - Welche Daten dürfen nicht entstehen?
 - **Student** ohne **Immatrikulation**
 - **Student** ohne **MatrNr**

Designprinzipien

Anwendungstreue - Semantik

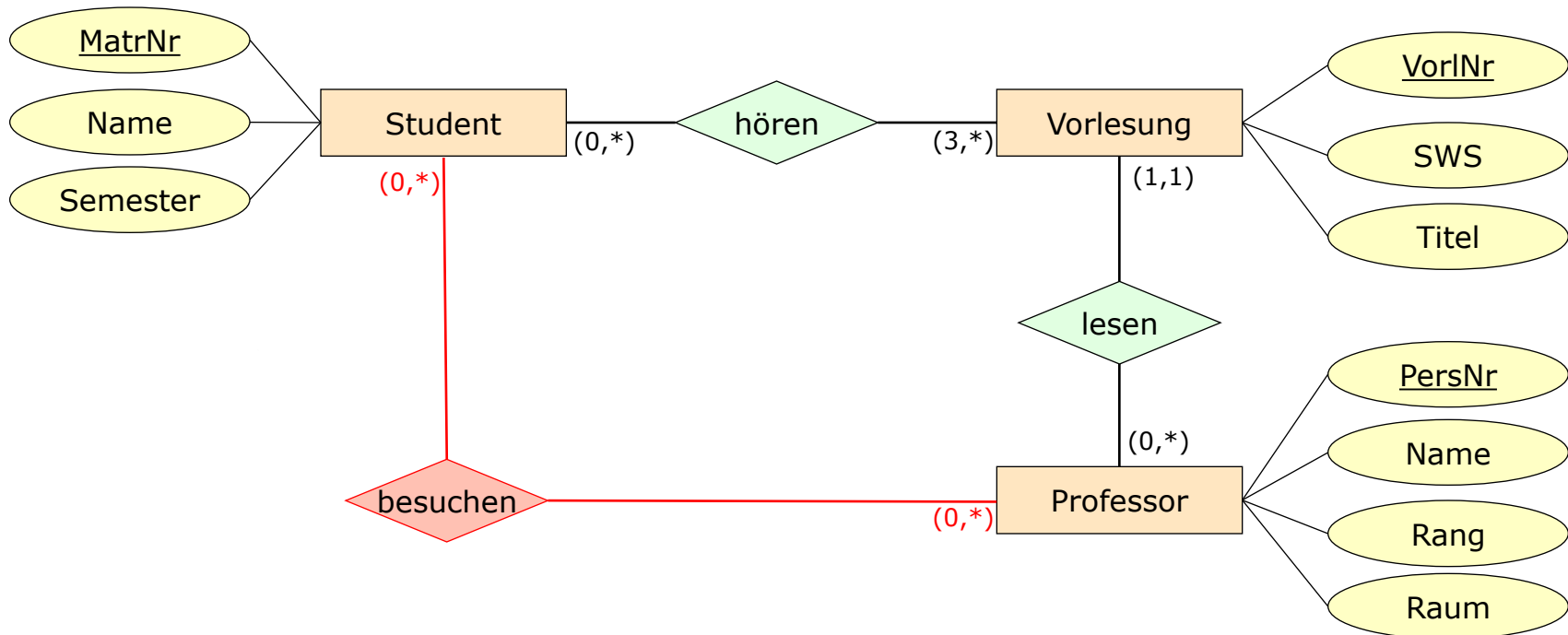
- Anwendungsfall: Vorlesung und Professor - je nach Semantik
 - Eine **Vorlesung** kann nur von einem **Professor** gehalten werden
 - **N:1**
 - Mehrere **Professoren** halten im Team (bspw. Co-Teaching) eine **Vorlesung** ab
 - **M:N**
 - Nicht nur aktuelle Vorlesungsabhaltung sondern auch Historie soll gespeichert werden
 - **M:N**

Designprinzipien

Vermeidung von Redundanz

■ Connection Trap (Verbindungsfall)

- Eine Falle in die man leicht fällt, ist die **Connection Trap** = redundante, zyklische Beziehungen
- **besuchen** ist redundant, sie wird bereits durch **hören** und **lesen** ausgedrückt

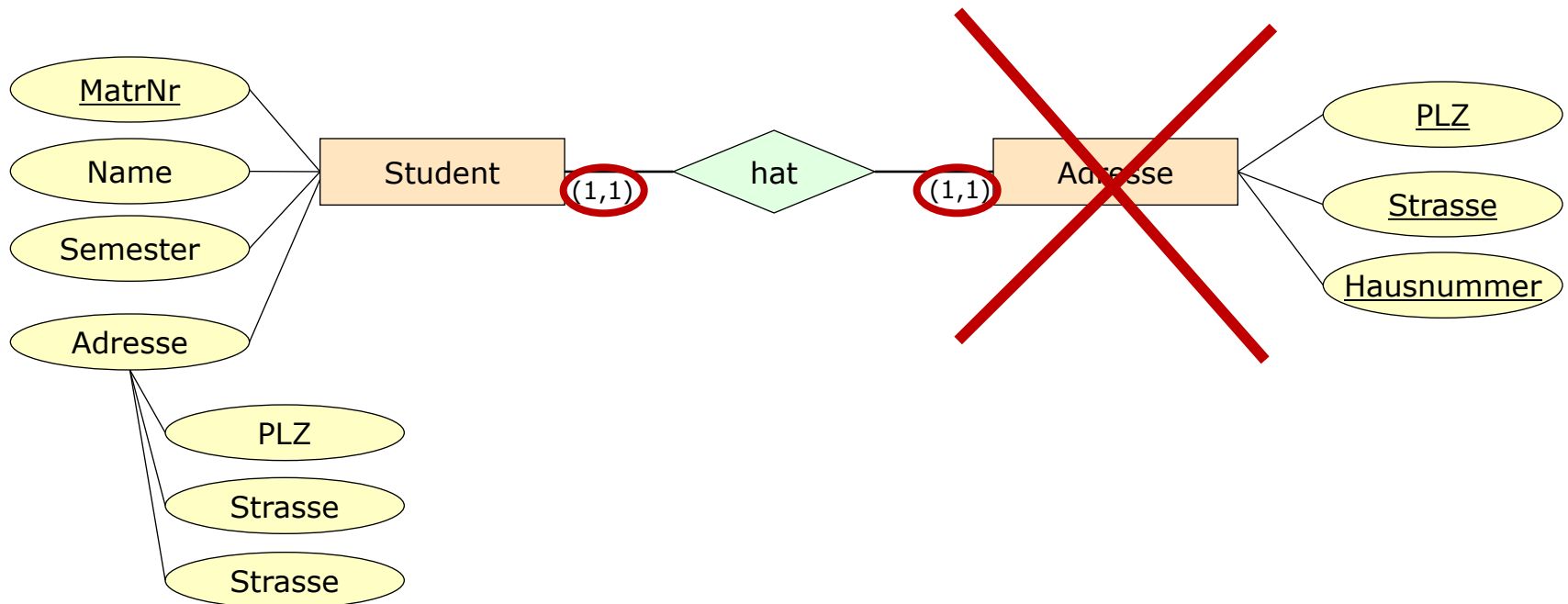


Designprinzipien

Einfachheit - Entitätstypen

■ KISS: Keep It Simple, Stupid

- Unnötige Verwendung von Entitätstypen vermeiden
- Ein **Student** hat eine **Adresse**



Designprinzipien

Entität versus Attribut

■ Entität versus Attribut

- Entitätstyp ist flexibler, Attribut einfacher zu implementieren
- Entitätstyp ist gerechtfertigt, falls
 - er mehr als nur den Namen einer Entität darstellt,
 - oder er den N-Teil eines 1:N-Beziehungstyps ist.
- Beispiel: Adressinformationen von Angestellte sollen gespeichert werden
 - Falls es mehrere Adressen pro Angestellten geben kann, modelliere Adresse als eigene Entität (da Attribute nicht mengenwertig sein dürfen)
 - Falls eine Adresse in einzelne Bestandteile zerlegt werden soll (Strasse, Hausnummer, PLZ, Stadt, usw.), dann zerlege Adresse in einzelne Attribute (diese können dann eventuell in eigene Entität ausgegliedert werden)

Designprinzipien

Schwache Entitätstypen

- Man scheut sich oft einen Schlüssel zu deklarieren.
- Die Folge - man schwächt einen Entitätstypen und macht alle seine Beziehungstypen zu identifizierenden Beziehungstypen.
- In der Realität werden oft künstliche Schlüssel eingesetzt
 - **MatrNr**, **SVNR**, **ISBN-10**, etc.
- Grund für das Fehlen eines solchen Schlüssels: Es gibt keine entsprechende Autorität, die einen solchen Schlüssel vergeben könnte.
 - Bspw. ist es unwahrscheinlich, dass jeder/m Professor*in der Welt eine eindeutige **PersNr** zugewiesen wird.

Designprinzipien

Natürliche und künstliche Schlüssel

- Natürlicher Schlüssel (engl.: natural key)
 - Login- oder Account-Tabelle: E-Mail-Adresse
 - Personentabelle: Personal- oder Versicherungsnummer
 - Patienten in einer Arztpraxis: Sozialversicherungsnummer
 - Studenten auf Hochschulen: Matrikelnummer
 - Artikelnummer: Global/European Article Number
 - Büchertabelle: ISBN/ISSN

- Künstlicher Schlüssel (engl.: surrogate key)
 - kein inhaltlicher Zusammenhang mit den Daten
 - einfachster Fall ist durchlaufende Nummer

Designprinzipien

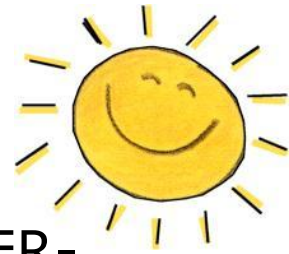
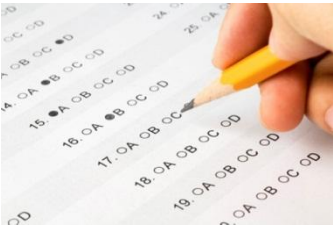
Natürliche versus künstliche Schlüssel

■ Argumente für künstliche Schlüssel

- Eindeutigkeit: numerische Schlüssel können fortlaufend vom DBMS generiert werden
- Effizienz und Einfachheit: kompakte Schlüssel sind platzsparend und effizient bei Verbundoperationen

■ Argumente für natürliche Schlüssel

- Authentizität: Schlüssel stehen in Zusammenhang mit den Daten
- Speicherplatz: es muss kein zusätzliches Attribut eingeführt werden
- Keine überflüssigen Indizes
- DBMS-unabhängig
- Keine unerkannten Doppelgänger



Welche der folgenden Punkte sind Designprinzipien der ER-Modellierung?

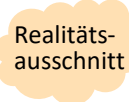
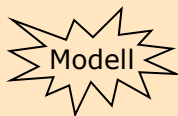
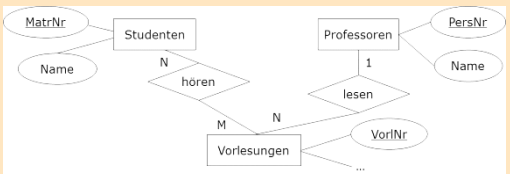
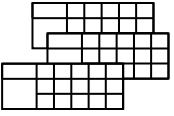
- ☒ Sparsamkeit mit Beziehungstypen
- ☐ Niedrige Kardinalitäten
- ☐ Sparsamkeit mit Schlüsseln
- ☒ Redundanzvermeidung
- ☒ Anwendungstreue
- ☒ Einfachheit

Zusammenfassung

ER-Diagramme

- ER-Diagramme sind eine **weit verbreitete** Art der konzeptuellen Modellierung
- Die Konzepte sind **nicht allzu schwer zu begreifen**, d.h. der Sachverhalt kann auch mit "Informatiklaien" (die den Fachbereich kennen) diskutiert werden
- Prinzipiell „**mit Papier und Bleistift**“ verwendbar, doch auch von Werkzeugen unterstützt
- Wegen Einschränkungen vermehrt Verdrängung der ER-Diagramme durch **objektorientierte Modelle (UML)**

Datenbankentwurfsschritte

Phase	Modell	Ergebnis
Anforderungsanalyse	Requirements Engineering	Pflichtenheft (Use-Cases, ...)  ANF 5.4: Eine Vorlesung wird durch eine eindeutige Vorlesungsnummer identifiziert. ...
Konzeptueller Entwurf	Entity Relationship (UML, ...)	Konzeptuelles DB-Schema  
Logischer Entwurf	Datenbankmodell (hierarchisch, objekt-orientiert, objekt-relational, semi-strukturiert, ...)	Logisches DB-Schema  <pre> Vorlesung{<u>VorlNr</u>, Titel, ...} Student{<u>MatrNr</u>, Name, ...} </pre>
Physischer Entwurf	Data Definition Language	Physisches DB-Schema  <pre> CREATE TABLE Vorlesung(VorlNr NUMBER(10) PRIMARY KEY, Titel VARCHAR(20), ...); </pre>

Anhang I: Definitionen

Begriff	Informale Bedeutung
Entity/Entität	zu repräsentierende Informationseinheit
Entitätstyp	Gruppierung von Entitäten mit gleichen Eigenschaften (= Entitätsmengen)
Beziehungstyp	Gruppierung von Beziehungen zwischen Entitäten
Attribut	datenwertige Eigenschaft einer Entität oder einer Beziehung
Schlüssel	identifizierende Eigenschaft von Entitäten, sind eine minimale Menge von Attributen
Kardinalität	Einschränkung von Beziehungstypen bezüglich der mehrfachen Teilnahme von Entitäten an der Beziehung
Stelligkeit	Anzahl der an einem Beziehungstyp beteiligten Entitätstypen
funktionale Beziehung	Beziehungstyp mit Funktionseigenschaft
is-a-Beziehung	Spezialisierung von Entitätstypen
Rollen	Beschreibung wie die an einer Beziehung beteiligten Entitäten sich verhalten
Schwache Entitäten	Entitäten, deren Existenz von einer anderen, übergeordneten Entitäten abhängen und die durch eine Kombination mit dem Schlüssel der übergeordneten Entität identifizierbar sind.